

Klaudia Wiktoria Malesa

Numer albumu: x

**Projekt i implementacja urządzenia do sortowania obiektów
według barwy w oparciu o technologię IoT.**

Praca dyplomowa
na studiach I-go stopnia
na kierunku Informatyka

Promotor pracy dyplomowej:

x

Katedra Systemów Informatycznych

Kielce 2026

Projekt i implementacja urządzenia do sortowania obiektów według barwy w oparciu o technologię IoT.

Streszczenie

Celem pracy było zaprojektowanie i wykonanie zautomatyzowanego urządzenia mechatronicznego do segregacji obiektów na podstawie barwy. Główny układ sterujący oparto na mikrokontrolerze Arduino Nano oraz czujniku TCS34725, a konstrukcję mechaniczną i elektroniczną opracowano w środowiskach AutoCAD i KiCad. Zaimplementowany algorytm analizy proporcji barw zapewnia wysoką skuteczność procesu sortowania. Przeprowadzone badania weryfikacyjne potwierdziły stabilność pracy układu w zmiennych warunkach oświetleniowych, osiągając sprawność klasyfikacji mechanicznej na poziomie 98%. Funkcjonalność stanowiska rozszerzono o dedykowaną aplikację w języku Python, umożliwiającą wizualizację pracy sortera oraz archiwizację danych pomiarowych.

Słowa kluczowe: automatyka, mechatronika, sorter kolorów, Arduino Nano, TCS34725, wizualizacja danych

Design and implementation of a device for sorting objects by color based on IoT technology.

Summary

The aim of the thesis was to design and construct an automated mechatronic device for sorting objects based on color. The main control system is based on an Arduino Nano microcontroller and a TCS34725 sensor, while mechanical and electronic designs were developed in AutoCAD and KiCad. The implemented color proportion analysis algorithm ensures high sorting efficiency. Verification tests confirmed the stability of the system under changing lighting conditions, achieving a mechanical classification efficiency of 98%. The functionality of the station was extended with a dedicated Python application, enabling process visualization and measurement data archiving.

Keywords: automation, mechatronics, color sorter, Arduino Nano, TCS34725, data visualization

Spis treści

WSTĘP.....	4
1. ANALIZA ISTNIEJĄCYCH ROZWIĄZAŃ URZĄDZEŃ SORTUJĄCYCH OBIEKTY WEDŁUG BARWY.....	7
1.1. Przegląd i analiza wiodących rozwiązań przemysłowych.....	7
1.2. Rozwiązania kompaktowe i modułowe (Mid-scale).....	11
1.3. Rozwiązania małoskalowe i edukacyjne.....	13
2. CHARAKTERYSTYKA TECHNOLOGII ORAZ CZUJNIKÓW KOLORU I ELEMENTÓW WYKONAWCZYCH WYKORZYSTYWANYCH W PROJEKCIE.....	15
2.1. Mikrokontroler Arduino Nano jako moduł zarządzający układem.....	15
2.2. Cyfrowy czujnik barwy TCS34725.....	17
2.3. Mechanizm wykonawczy - serwomechanizmy MG-90S.....	19
2.4. Interfejs wizualizacyjny - wyświetlacz LCD 16x2 z konwerterem I2C.....	21
2.5. Układ zasilania systemu.....	23
3. OPRACOWANIE KONCEPCJI ORAZ PROJEKTU MECHANICZNEGO I ELEKTRONICZNEGO URZĄDZENIA SORTUJĄCEGO.....	25
3.1. Założenia projektowe i dobór surowca sortowanego.....	25
3.2. Koncepcja działania i logika procesu.....	27
3.3. Projektowanie mechaniczne w środowisku AutoCAD.....	30
3.4. Projekt układu elektronicznego w środowisku KiCad.....	34
4. IMPLEMENTACJA UKŁADU STERUJĄCEGO I OPROGRAMOWANIA REALIZUJĄCEGO PROCES SORTOWANIA OBIEKTÓW WEDŁUG BARWY.....	40
4.1. Budowa obudowy układu.....	40
4.2. Program do kalibracji i analizy surowych danych.....	45
4.3. Analiza danych pomiarowych.....	47
4.4. Opracowanie i analiza algorytmu sterującego w środowisku Arduino IDE.....	50
4.5. Finalizacja i wygląd sortera.....	58
5. INTEGRACJA UKŁADU STERUJĄCEGO Z MODUŁEM PREZENTACJI WYNIKÓW DZIAŁANIA URZĄDZENIA.....	61
5.1. Architektura obliczeń krawędziowych (Edge Computing) w systemie IoT.....	62
5.2. Rozbudowa oprogramowania Arduino o moduł transmisji danych.....	63
5.3. Struktura i analiza modułowa skryptu Python.....	64
5.4. Dokumentacja wizualna modułu prezentacji.....	68

6. TESTOWANIE I OCENA DZIAŁANIA WYKONANEGO URZĄDZENIA ORAZ ANALIZA WYNIKÓW PRACY SYSTEMU.....	71
6.1. Testy funkcjonalne systemu.....	72
6.2. Testy kalibracyjne i weryfikacja stabilności czujnika koloru.....	73
6.3. Ocena sprawności mechanicznej układu rozdzielającego.....	75
6.4. Analiza specyfiki detekcji koloru czerwonego i identyfikacja błędów klasyfikacji.....	76
6.5. Weryfikacja spójności danych i niezawodności transmisji UART.....	77
PODSUMOWANIE.....	78
LITERATURA.....	80
ZAŁĄCZNIKI.....	83
Spis rysunków.....	83
Spis listingów.....	85
Spis tabel.....	85

WSTĘP

Rozwój automatyzacji stanowi obecnie jeden z kluczowych kierunków zmian w inżynierii produkcji i jest ściśle powiązany z koncepcją Przemysłu 4.0 [12]. Pojęcie to opisuje czwartą rewolucję przemysłową opartą na powszechnej cyfryzacji. Jej fundamentem są systemy cyberfizyczne, które polegają na wyposażaniu maszyn w zaawansowane układy komunikacyjne oraz sensory. Współczesne zakłady coraz chętniej wdrażają rozwiązania ograniczające udział człowieka w zadaniach powtarzalnych, które wymagają dużej precyzji. Taka tendencja wynika z rosnących wymagań rynkowych względem jakości wyrobów oraz narzuconego tempa realizacji procesów. Automatyzacja pozwala nie tylko redukować liczbę błędów, ale też realnie poprawia efektywność i bezpieczeństwo na stanowiskach pracy.

W ostatnich latach można zaobserwować duży postęp w dziedzinie tanich mikrokontrolerów oraz popularnych elementów wykonawczych. Szczególne znaczenie miało upowszechnienie platformy Arduino, która stała się standardem w projektach amatorskich i naukowych. Dzięki niej projektowanie systemów stało się dostępne dla szerokiego grona osób, co sprzyja nauce automatyki w sposób praktyczny. Takie podejście ułatwia szybkie prototypowanie oraz testowanie założeń konstrukcyjnych przed podjęciem decyzji o budowie większej, przemysłowej instalacji [13]. Proces ten występuje w niemal każdej branży i decyduje o sprawnej organizacji przepływu materiałów - od prostego rozdzielania elementów według wielkości, po zaawansowaną klasyfikację na podstawie barwy. Tradycyjne sortowanie ręczne uznaje się za mało wydajne, a M.P. Groover [7] słusznie zauważa, że zastąpienie pracy ludzkiej automatami eliminuje uciążliwą rutynę, co bezpośrednio przekłada się na wyższą jakość produktu końcowego. Maszyny mogą pracować w trybie ciągłym, zachowując przy tym identyczne parametry pracy przez wiele godzin.

Kolor jest parametrem wyjątkowo często wykorzystywanym w procesach selekcji produktów. Dostarcza on bowiem szybkiej informacji o stanie lub rodzaju obiektu bez konieczności nawiązywania kontaktu fizycznego. W przemyśle spożywczym pozwala ocenić stopień dojrzałości owoców, natomiast w zakładach recyklingu umożliwia sprawne rozróżnianie surowców. Metoda ta pomaga błyskawicznie wykrywać przebarwienia lub niepożądane wady powierzchniowe, a współczesne urządzenia sortujące osiągają skuteczność na poziomie nawet 91–98% [16]. Skuteczne rozpoznawanie barw wymaga jednak zapewnienia stabilnych i powtarzalnych warunków pomiaru. Z tego powodu konstrukcja sortera musi przewidywać sztywne zamocowanie sensora w ściśle określonej odległości od celu oraz osłony mechaniczne, które odcinają dopływ światła

z otoczenia. Ważnym etapem jest również kalibracja programowa, która dostosowuje układ do pracy w konkretnym środowisku [11].

Automatyczne sortowanie to nie tylko pomiar, lecz także wykonanie właściwej reakcji. W mniejszych i średnich urządzeniach często stosuje się serwomechanizmy, ponieważ są proste w sterowaniu, szybkie i zapewniają dużą dokładność ruchu. Sercem układu pozostaje mikrokontroler, który odbiera dane z czujników, realizuje algorytm sterowania i wydaje polecenia elementom wykonawczym. Zaletą tego typu rozwiązań jest elastyczność, gdyż oprogramowanie można łatwo modyfikować, dostosowując system do nowych zadań [18]. W nowoczesnych układach dużą wagę przykładają się także do monitorowania działania urządzeń. Możliwość obserwowania pracy systemu w czasie rzeczywistym ułatwia wykrywanie błędów i podnosi bezpieczeństwo użytkowania [9].

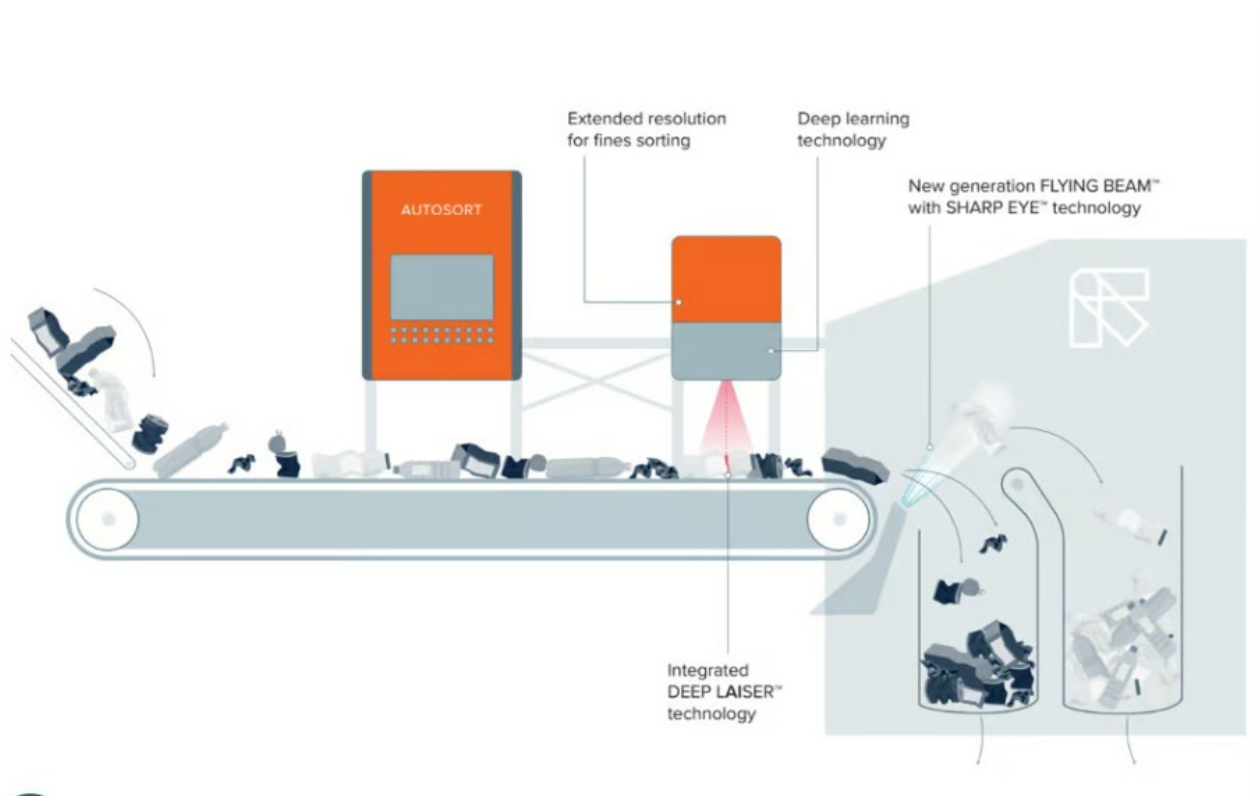
Na tle opisanych zagadnień pojawia się potrzeba stworzenia niewielkiego urządzenia, które pokazuje, w jaki sposób można automatycznie sortować obiekty według barwy przy użyciu prostych podzespołów elektronicznych. Celem niniejszej pracy jest zaprojektowanie i wykonanie prototypowego sortera kolorów opartego na mikrokontrolerze Arduino. Urządzenie ma rozpoznawać barwę badanego obiektu za pomocą czujnika koloru, a następnie kierować go do odpowiedniej przegrody z użyciem serwomechanizmów. Dodatkowo urządzenie zostanie wyposażone w wyświetlacz LCD, który umożliwi prezentację podstawowych informacji o pracy systemu. Ponadto konstrukcja umożliwi dwukierunkową komunikację z komputerem, co pozwoli na przesyłanie parametrów pracy i wizualizację zbieranych danych w czasie rzeczywistym. W celu praktycznej weryfikacji poprawności działania rozpoznawania barw, jako modelowe obiekty testowe wybrano cukierki typu Skittles. Cukierki te charakteryzują się nasyconymi i łatwo rozróżnialnymi barwami, co pozwoli na precyzyjne skalibrowanie czujnika koloru oraz sprawdzenie skuteczności algorytmu sterującego serwomechanizmami w warunkach zbliżonych do rzeczywistych procesów sortowania.

Zakres pracy obejmuje analizę istniejących rozwiązań stosowanych w sorterach kolorów, przygotowanie projektu mechanicznego i elektronicznego urządzenia, opracowanie oprogramowania odpowiedzialnego za sterowanie oraz przeprowadzenie testów działania zbudowanego prototypu. Ocenie poddana zostanie skuteczność działania urządzenia oraz stabilność poszczególnych etapów procesu sortowania.

1. ANALIZA ISTNIEJĄCYCH ROZWIĄZAŃ URZĄDZEŃ SORTUJĄCYCH OBIEKTY WEDŁUG BARWY

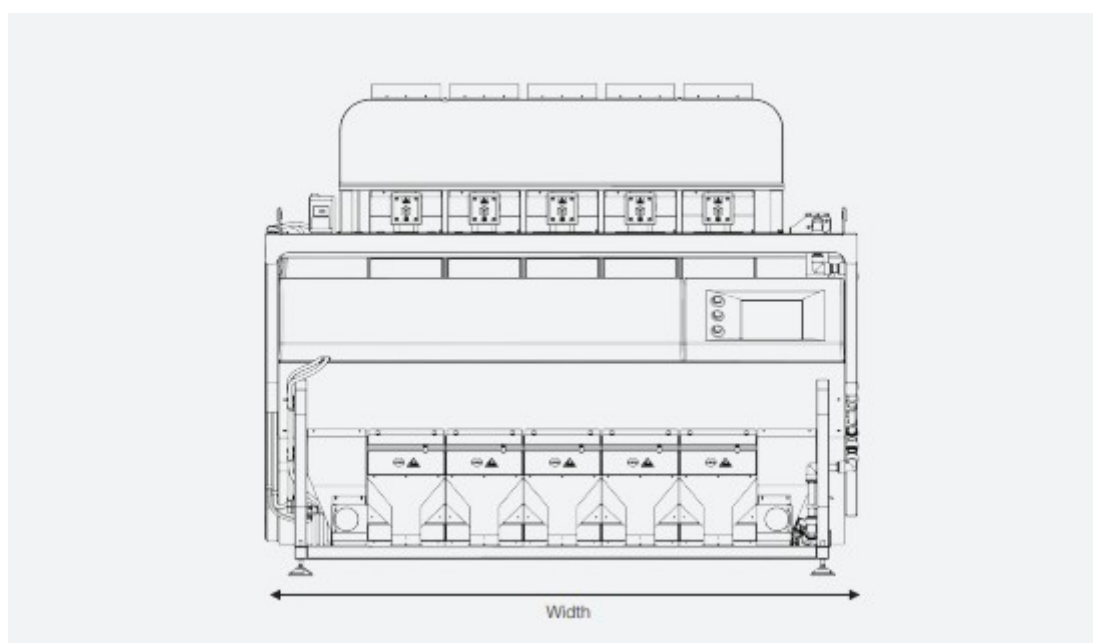
1.1. Przegląd i analiza wiodących rozwiązań przemysłowych.

Obecnie na rynku automatyki przemysłowej dominuje kilka firm, których rozwiązania wyznaczają standardy w klasyfikacji produktów. Największym światowym liderem jest norweska firma TOMRA Sorting, która specjalizuje się w zaawansowanych systemach wizyjnych wspieranych przez sztuczną inteligencję. Ich urządzenia są powszechnie stosowane w recyklingu do rozpoznawania plastiku czy szkła, a także w rolnictwie do selekcji owoców i warzyw. Technologia tej firmy opiera się na fuzji danych z kamer RGB, czujników laserowych oraz kamer NIR, czyli bliskiej podczerwieni. Do oddzielania obiektów od głównego nurtu produkcji wykorzystują one bardzo szybkie systemy pneumatyczne, które wypychają dany element precyzyjnym strumieniem powietrza [21].



Rys. 1.1.1 Przemysłowy system sortowania optycznego AUTOSORT [Źródło: TOMRA Systems ASA, 2024]

Kolejną firmą która odgrywa istotną rolę w tej dziedzinie jest Bühler Group [4], uznawana za wzór w dziedzinie technologii spożywczych. Ich sortery z serii SORTEX są standardem w młynach i liniach produkcyjnych na całym świecie, gdzie służą do analizy barwy i kształtu ziaren ryżu, pszenicy czy kukurydzy. Firma ta słynie z używania precyzyjnych kamer CCD, które pozwalają na zachowanie sporej wydajności przy jednoczesnym wykrywaniu nawet najmniejszych przebarwień produktów. Podobną koncepcję separacji produktów sypkich realizuje japońska firma Satake [20]. Ich systemy charakteryzują się wykorzystaniem precyzyjnych dysz pneumatycznych, które pozwalają na skuteczne oddzielanie zanieczyszczonych ziaren od pełnowartościowego surowca, co jest kluczowe dla zachowania wysokich standardów czystości w przemyśle rolniczym.

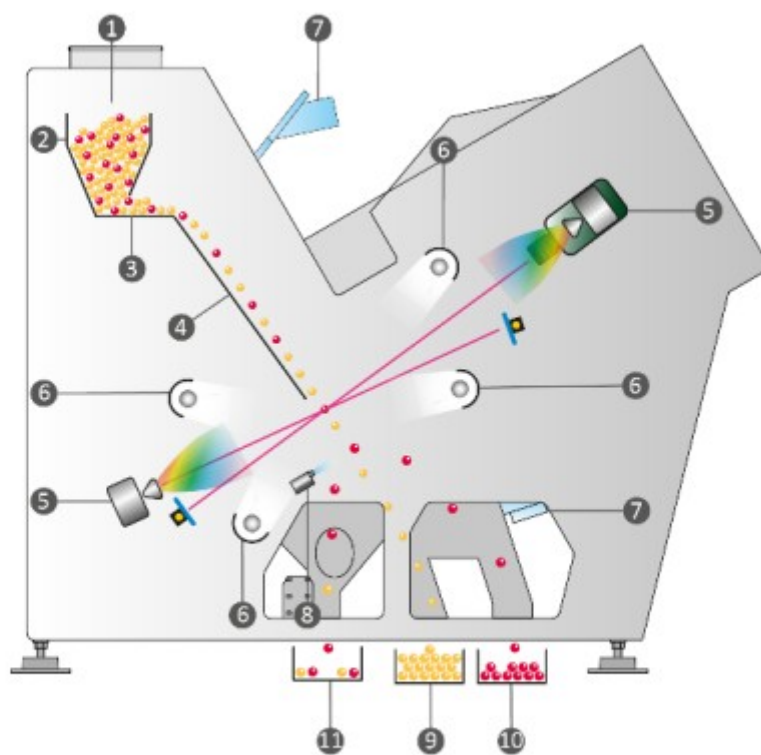


Rys. 1.1.2. Widok techniczny sortera optycznego z serii SORTEX A [Źródło: Bühler Group, 2023]



Rys. 1.1.3. Przemysłowy sorter optyczny wykorzystujący wysokociśnieniowe dysze pneumatyczne [Źródło: Satake Corporation, 2024]

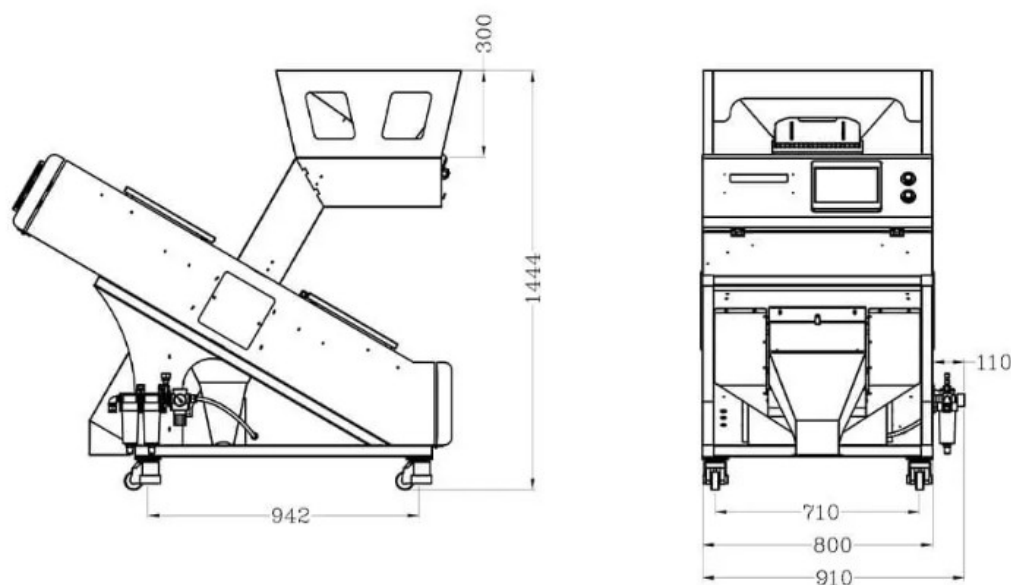
Warto również zwrócić uwagę na amerykańską firmę Key Technology, która od ponad 70 lat projektuje systemy dla przemysłu spożywczego, łącząc transport wibracyjny z precyzyjną selekcją cyfrową. Ich maszyny potrafią jednocześnie analizować kolor, kształt oraz strukturę produktu, a dzięki wykorzystaniu laserów i oświetlenia UV wykrywają ciała obce z ogromną wydajnością, przekraczającą 20 ton na godzinę [10]. Takie parametry są niezbędne przy produkcji mrożonek czy bakalii, gdzie jakość i bezpieczeństwo są priorytetem. Podobne rozwiązania, ale do pracy w znacznie cięższych warunkach, oferuje austriacka firma Binder+Co, której system „CLARITY” pozwala na ciągłą segregację szkła z recyklingu oraz surowców mineralnych [2]. Ofertę tych największych systemów przemysłowych uzupełnia firma Cimbria z serią sorterów SEA (Rys. 1.1.4.). Maszyny te stosują metodę kaskadową, co oznacza, że produkt jest prześwietlany z obu stron przez zaawansowane kamery i może być sortowany wielokrotnie w jednym urządzeniu, aby uzyskać jak najwyższą czystość towaru.



Rys. 1.1.4. Schemat wewnętrzny i zasada działania kaskadowego sortera optycznego serii SEA [Źródło: Cimbria, 2025]

1.2. Rozwiązania kompaktowe i modułowe

Poza ogromnymi maszynami, na rynku znajdziemy też sortery typu Mid-scale, które są przeznaczone dla nieco mniejszych zakładów, takich jak lokalne palarnie kawy czy mniejsze gospodarstwa rolne. Dobrym przykładem takiego rozwiązania jest model dedykowany do sortowania ziaren kawy firmy WESORT, który ma około półtora metra wysokości i posiada jedną rynną zsypową. Choć urządzenie to jest znacznie mniejsze od systemów oferowanych przez liderów rynku, wciąż posiada wszystkie najważniejsze funkcje nowoczesnego sprzętu, w tym systemy wizyjne wspierane przez algorytmy sztucznej inteligencji, które pomagają w automatycznym rozpoznawaniu cech produktów. Wewnątrz maszyny pracują kamery RGB o wysokiej rozdzielczości, które analizują każde ziarno, a za odrzucanie wadliwych elementów odpowiadają szybkie dysze pneumatyczne. Urządzenie jest wyposażone we własny ekran dotykowy, co pozwala na łatwą zmianę ustawień i podgląd pracy w czasie rzeczywistym bez potrzeby używania zewnętrznych komputerów. Dzięki swojej pionowej konstrukcji sorter ten zajmuje mniej miejsca i zapewnia wysoką precyzję, będąc jednocześnie znacznie tańszym w eksploatacji niż potężne linie przemysłowe. Rozwiązania tego typu pokazują, że profesjonalną technologię optyczną można skutecznie zamknąć w bardziej kompaktowej formie, co czyni ją dostępną dla szerszego grona użytkowników [24].



Rys. 1.2.1. Rysunek zestawieniowy sortera optycznego WESORT z określeniem głównych wymiarów gabarytowych [Opracowanie na podstawie dokumentacji technicznej producenta].

Kolejnym przykładem urządzenia, które warto przytoczyć w kontekście rozwiązań kompaktowych, jest model Meyer M2 (MSXC-120M) produkowany przez firmę Meyer Europe. Urządzenie to zostało zaprojektowane z myślą o rolnikach, spółdzielniach oraz mniejszych zakładach przetwórczych, które wymagają profesjonalnej jakości separacji przy jednoczesnym ograniczeniu zajmowanej powierzchni. Maszyna charakteryzuje się wysokością około 1,5 metra oraz masą 300 kg, co klasyfikuje ją jako jednostkę wolnostojącą o wysokiej mobilności, mogącą pracować z wydajnością od 0,4 do 2,4 tony na godzinę. Pomimo mniejszej skali, model ten wykorzystuje zaawansowane systemy wizyjne oparte na kamerach HD, które skanują materiał z prędkością 50 000 razy na sekundę, co pozwala na identyfikację defektów o minimalnej powierzchni 0,0025 mm². Kluczowym elementem technologicznym jest zastosowanie algorytmów sztucznej inteligencji wspierających głębokie uczenie, które automatycznie analizują barwę i kształt produktów, porównując je z obszernymi bibliotekami modeli. Całość systemu uzupełnia intuicyjny interfejs dotykowy oraz funkcja zdalnego sterowania, co sprawia, że profesjonalna technologia sortowania o skuteczności przekraczającej 99,9% staje się dostępna dla podmiotów nieposiadających rozbudowanej infrastruktury przemysłowej [17].

1.3. Rozwiązania małoskalowe i edukacyjne

Osobną grupę stanowią systemy laboratoryjne oraz zestawy edukacyjne, które pozwalają na naukę podstaw automatyki i przetwarzania sygnałów w niskiej cenie. Bardzo popularnym przykładem w środowisku akademickim jest zestaw LEGO Mindstorms EV3, który umożliwia budowę w pełni funkcjonalnych systemów sortujących kolory. Mechanizm działania takiego urządzenia opiera się na wykorzystaniu czujnika koloru, który skanuje przesuwające się obiekty i przesyła dane o ich barwie do jednostki sterującej. Po rozpoznaniu konkretnego koloru, program uruchamia odpowiedni silnik (serwomechanizm), który fizycznie przesuwa lub spycha element do właściwego pojemnika. Budowa takich systemów wymaga od projektanta stworzenia sprawnego mechanizmu podającego oraz logicznego powiązania odczytów z czujnika z ruchem elementów wykonawczych [14].



Rys. 1.3.1. Model edukacyjny systemu sortującego kolory oparty na platformie LEGO Mindstorms EV3. Źródło: LEGO Education, 2024.

Równolegle do zestawów klockowych, hobbystów i studentów inspirują projekty oparte na platformie Arduino. Wykorzystują one podobną logikę działania, najczęściej z użyciem czujników z serii TCS. Charakteryzują się tym, że dają one znacznie większą swobodę w projektowaniu własnej elektroniki. Choć konstrukcje edukacyjne mają znacznie mniejszą wydajność niż profesjonalne sortery przemysłowe, doskonale udowadniają, że logikę selekcji optycznej można zrealizować w formie niewielkiego urządzenia biurkowego. Inspirując się tymi koncepcjami oraz dążąc do stworzenia taniego, a jednocześnie profesjonalnie wykonanego systemu, w ramach niniejszej pracy postanowiono zaprojektować i zbudować własny sorter optyczny. Wykorzystanie platformy Arduino wraz z dedykowaną płytką PCB pozwala na stworzenie systemu, który łączy prostotę rozwiązań edukacyjnych z funkcjonalnością zbliżoną do profesjonalnych urządzeń laboratoryjnych.

2. CHARAKTERYSTYKA TECHNOLOGII ORAZ CZUJNIKÓW KOLORU I ELEMENTÓW WYKONAWCZYCH WYKORZYSTYWANYCH W PROJEKCIE.

2.1. Mikrokontroler Arduino Nano jako moduł zarządzający układem

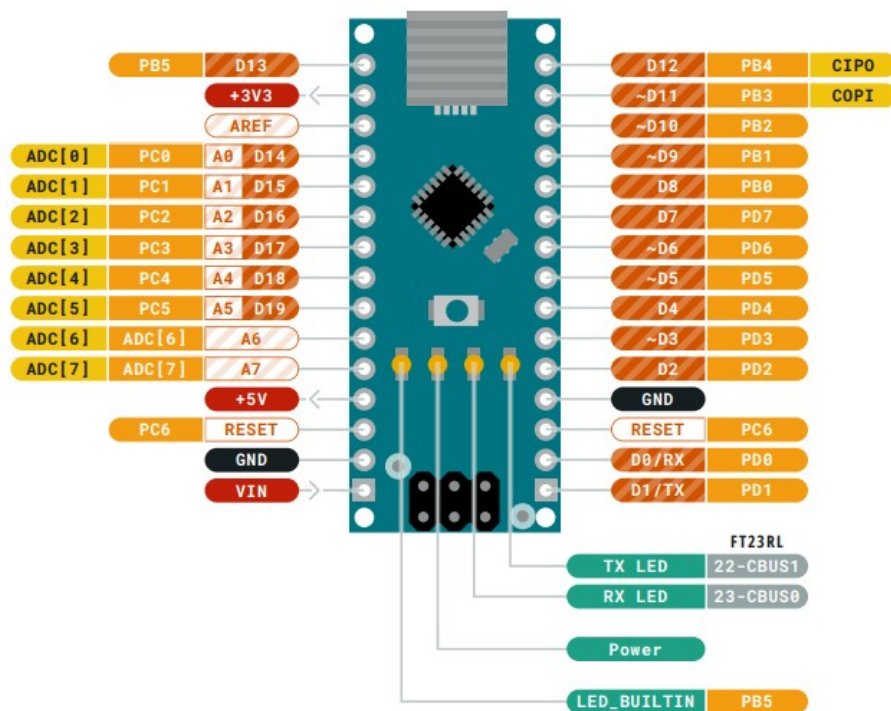
W nowoczesnych sorterach technologia Internetu Rzeczy – IoT - opiera się na integracji systemowej wszystkich czujników oraz silników, co umożliwia inteligentne reagowanie na zdarzenia zachodzące na linii produkcyjnej. W niniejszym projekcie koncepcję tę zrealizowano z wykorzystaniem mikrokontrolera Arduino Nano, który pełni funkcję jednostki centralnej zarządzającej pracą całego układu. Jest to niewielka płytką rozwojowa wyposażona w 8-bitowy mikrokontroler ATmega328P. Wybór ten był podyktowany faktem, że pomimo miniaturowych rozmiarów, moduł oferuje odpowiednią liczbę pinów wejścia/wyjścia, co pozwala na jednoczesną obsługę czujnika koloru, dwóch serwomechanizmów oraz wyświetlacza LCD.

Arduino Nano działa jak „mózg” całego procesu. Mikrokontroler ten jest wyposażony w pamięć Flash, gdzie zapisywany jest program; pamięć SRAM do bieżących obliczeń oraz taktowanie zegarem o częstotliwości 16 MHz. W praktyce oznacza to, że urządzenie potrafi wykonać miliony operacji w ciągu sekundy. Dzięki temu realizuje koncepcję Edge Computing - analizuje sygnały z czujnika TCS34725 natychmiast po ich odebraniu. Zamiast przysyłać dane do zewnętrznych systemów, Arduino samodzielnie „podejmuje decyzję” o tym, jaki kolor widzi i od razu wysyła sygnał sterujący do mechanizmów.

Kluczem do poprawnego działania całego układu jest odpowiednie wykorzystanie wyprowadzeń płytki, które zostały przedstawione na schemacie Pinout (Rys. 2.1.1.). Arduino Nano posiada piny cyfrowe, oznaczone jako D, oraz analogowe - oznaczone jako A. Szczególnie ważne są piny A4 (SDA) oraz A5 (SCL), ponieważ tworzą one magistralę I2C. To właśnie dzięki nim mikrokontroler może komunikować się jednocześnie z czujnikiem koloru i wyświetlaczem, używając tylko dwóch przewodów sygnałowych. Do sterowania serwami wykorzystane zostaną z kolei piny z funkcją PWM, które pozwalają na precyzyjne ustawienie kąta wychylenia ramienia sortującego.

Ważnym aspektem technicznym Arduino Nano jest wbudowany port micro-USB, który pozwala na łatwe programowanie i zasilanie płytki bezpośrednio z komputera podczas testów.

Płytkę posiada także wbudowany stabilizator napięcia, co pozwala na zasilanie jej napięciem od 7 V do 12 V przez pin VIN. Jest to wiedza niezbędna przy projektowaniu własnej płytki PCB, aby zapewnić stabilne zasilanie dla wszystkich podzespołów. Dzięki tak zaplanowanej architekturze, gotowy sorter, mimo niewielkich rozmiarów, będzie działał na podobnych zasadach, co zaawansowane systemy przemysłowe stosowane w dużych fabrykach [1].



Rys. 2.1.1. Widok modułu Arduino Nano wraz z rozmieszczeniem pinów wejścia/wyjścia.

2.2. Cyfrowy czujnik barwy TCS34725

Głównym elementem odpowiedzialnym za rozpoznawanie kolorów w projekcie jest zaawansowany sensor TCS34725. Wybór tego modelu podyktowany został obecnością wbudowanego przetwornika analogowo-cyfrowego ADC, co odróżnia go od tańszych zamienników. Dzięki temu rozwiązaniu mikrokontroler otrzymuje przetworzone dane bezpośrednio przez magistralę I2C, co eliminuje konieczność czasochłonnego zliczania impulsów przez jednostkę centralną. Sercem czujnika jest matryca fotodiod RGBC, która poza kanałami czerwonym, zielonym i niebieskim, posiada dodatkowy kanał „Clear”. Pozwala on na precyzyjną ocenę natężenia oświetlenia otoczenia.

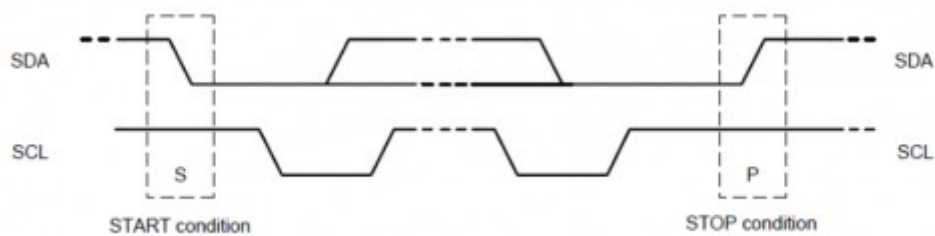
Istotną zaletą sensora jest zintegrowany filtr IR, który skutecznie blokuje promieniowanie podczerwone. Ma to kluczowe znaczenie w warunkach laboratoryjnych i domowych, gdzie standardowe oświetlenie emituje znaczne ilości podczerwieni mogącej zakłócać odczyty. Zastosowanie tego filtra sprawia, że zbierane dane są wiarygodne i zbliżone do spektrum widzialnego przez ludzkie oko, co zwiększa skuteczność rozpoznawania obiektów o podobnych odcieniach. Dodatkowo, wysoka dynamika pracy sensora (3 800 000:1) umożliwia stabilną pracę sortera w zróżnicowanych warunkach oświetleniowych, zapobiegając nasyceniu czujnika przy intensywnym świetle.

Komunikacja z czujnikiem odbywa się za pomocą szeregowej magistrali I2C, wykorzystującej dwie linie sygnałowe: danych (SDA) oraz zegarową (SCL). Wykorzystanie tego protokołu pozwala na połączenie wielu podzespołów, takich jak czujnik i wyświetlacz, przy użyciu minimalnej liczby przewodów, co znacząco ułatwia projektowanie płytki PCB.

Transmisja danych przebiega według ściśle określonego protokołu, co przedstawiono na powyższym schemacie (Rys. 2.2.2). Proces inicjowany jest przez mikrokontroler sygnałem startu S, definiowanym jako zmiana stanu linii SDA z wysokiego na niski przy wysokim stanie SCL. Dane przesyłane są w 8-bitowych ramkach, po których następuje bit potwierdzenia ACK. Po zaadresowaniu czujnika pod adresem sprzętowym 0x29, możliwy jest odczyt lub zapis danych w jego rejestrach. Całość komunikacji kończona jest sygnałem stopu (P), który przywraca magistralę do stanu spoczynku [23].



Rys. 2.2.1. Moduł zaawansowanego czujnika koloru TCS34725 wyposażony w matrycę fotodiod RGBC oraz zintegrowany filtr IR



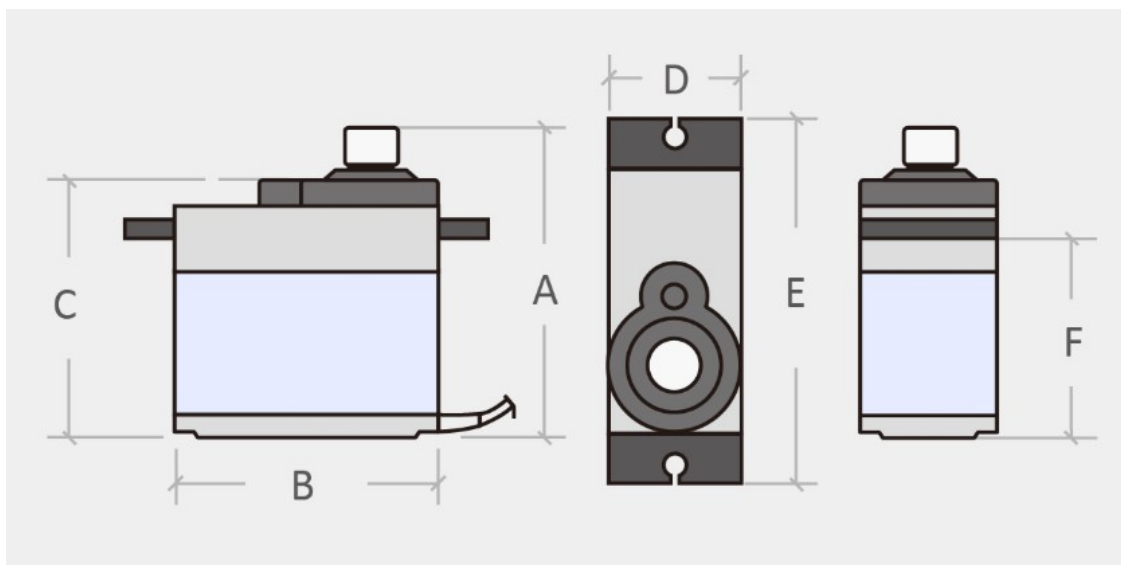
Rys. 2.2.2. Przebiegi czasowe sygnałów startu (S) oraz stopu (P) na liniach SDA i SCL magistrali I2C. [Źródło: opracowanie na podstawie Waveshare Wiki, 2024].

2.3. Mechanizm wykonawczy - serwomechanizmy MG-90S

Za fizyczną separację produktów w projekcie odpowiadają dwa serwomechanizmy typu micro, model MG-90S. Wybór tego komponentu podyktowany został jego niską masą oraz kompaktowymi wymiarami, co czyni go rozwiązaniem optymalnym dla konstrukcji o ograniczonej przestrzeni, opartych na platformie Arduino Nano. Zastosowanie plastikowych przekładni w wybranej konfiguracji pozwala na uniknięcie nadmiernego obciążenia konstrukcji nośnej obudowy, przy jednoczesnym zachowaniu momentu obrotowego wystarczającego do sprawnego przesuwania lekkich obiektów [22].

Zasada działania modelu MG-90S opiera się na koncepcji zamkniętej pętli sprzężenia zwrotnego. Urządzenie składa się z silnika prądu stałego, przekładni redukcyjnej o dużym przełożeniu oraz potencjometru, który pełni funkcję czujnika położenia wału wyjściowego. Układ elektroniczny serwomechanizmu nieustannie analizuje sygnał wejściowy i porównuje go z aktualną informacją z potencjometru. W przypadku wystąpienia różnicy (błędu pozycji), sterownik zasila silnik, aby skorygować położenie orczyka do momentu osiągnięcia stanu równowagi.

Kluczowym elementem sterującym jest sygnał *Pulse Width Modulation*, czyli modulacja szerokości impulsu. Mikrokontroler wysyła impulsy elektryczne o stałej częstotliwości, w których czas trwania stanu wysokiego definiuje konkretny kąt wychylenia osi. Przykładowo, impuls trwający 1 ms odpowiada jednej skrajnej pozycji, 1,5 ms ustawia wał w pozycji środkowej, a 2 ms przesuwa go do drugiej skrajnej pozycji. Taki sposób sterowania zapewnia bardzo wysoką powtarzalność ruchów oraz stabilność zadanego położenia, co jest niezbędne dla poprawnego funkcjonowania systemu segregacji produktów [8].



Rys. 2.3.1. Schemat wymiarowy serwomechanizmu MG-90S [Źródło: TowerPro, 2024]

2.4. Interfejs wizualizacyjny - wyświetlacz LCD 16x2 z konwerterem I2C

Kluczowym elementem systemu komunikacji z użytkownikiem w projekcie sortera jest alfanumeryczny wyświetlacz LCD 16x2. Zastosowany moduł z niebieskim podświetleniem umożliwia prezentację informacji w dwóch wierszach, z których każdy mieści do 16 znaków. Wykorzystanie tego komponentu pozwala na bieżące monitorowanie statystyk pracy urządzenia, takich jak liczba poprawnie posortowanych elementów czy aktualnie zidentyfikowana barwa obiektu. Pełni on również istotną funkcję diagnostyczną, ponieważ w przypadku wystąpienia awarii mechanizmu lub błędu odczytu danych z układu sensorycznego, na ekranie generowany jest stosowny komunikat, co pozwala na szybką lokalizację problemu bez konieczności analizy kodu programu przez zewnętrzny komputer [3].

Zasada działania wyświetlacza opiera się na sterowniku HD44780, który zarządza matrycą ciekłokrystaliczną. Każdy znak tworzony jest na siatce pikseli, a ich widoczność zależy od polaryzacji cząsteczek ciekłego kryształu pod wpływem przyłożonego pola elektrycznego. Ponieważ bezpośrednie sterowanie takim układem wymagałoby zajęcia wielu pinów cyfrowych mikrokontrolera Arduino Nano, w projekcie zastosowano konwerter oparty na układzie PCF8574.

Zastosowanie wspomnianego konwertera I2C pozwala na przesyłanie danych do wyświetlacza za pomocą szeregowej magistrali komunikacyjnej, wykorzystującej jedynie dwie linie sygnałowe: danych SDA oraz zegarową SCL. Układ PCF8574, zgodnie z dokumentacją techniczną, składa się z 8-bitowego rejestru dwukierunkowego, który umożliwia konwersję danych szeregowych z magistrali I2C na równoległe sygnały sterujące pinami danych i sterujących wyświetlaczem LCD. Dodatkowym atutem modułu jest zintegrowany potencjometr wieloobrotowy, służący do precyzyjnej, ręcznej regulacji kontrastu znaków. Pozwala to na optymalne dostosowanie czytelności wyświetlanych komunikatów do panujących warunków oświetleniowych [19].



Rys. 2.4.1. Widok tylnej części wyświetlacza LCD z zamontowanym konwerterem I2C opartym na układzie PCF8574.
Źródło: Botland.

2.5. Układ zasilania systemu

Istotnym aspektem na etapie doboru komponentów było zaplanowanie stabilnego systemu zasilania, zdolnego do obsłużenia obciążeń generowanych przez dwa serwomechanizmy MG-90S. Zdecydowano się na wykorzystanie modułu impulsowej przetwornicy DC-DC zintegrowanej z funkcją UPS - *Uninterruptible Power Supply*. Wybór stabilizowanego źródła prądu uzasadniony był faktem, iż serwomechanizmy podczas intensywnej pracy generują skokowe pobory prądu. Zastosowanie standardowego zasilania z portu USB komputera mogłoby prowadzić do krytycznych spadków napięcia, skutkujących resetowaniem się mikrokontrolera Arduino Nano lub błędami w odczytach czujnika kolorów.

Zasada działania wybranego modułu opiera się na technologii impulsowej konwersji napięcia. Przetwornica o wydajności prądowej do 3 A pozwala na efektywne obniżenie napięcia wejściowego do poziomu 5 V przy zachowaniu wysokiej sprawności energetycznej i niskiej emisji ciepła. Koncepcja zakłada doprowadzenie napięcia z zasilacza sieciowego do gniazda DC, skąd energia jest dystrybuowana do pinu VIN mikrokontrolera oraz bezpośrednio do szyn zasilających elementy wykonawcze.

Szczególną cechą wybranego rozwiązania jest funkcja UPS, która umożliwia bezprzerwowe przełączenie systemu na zasilanie z ogniwa litowego w przypadku zaniku napięcia w sieci głównej. Zintegrowany system ochrony ogniw monitoruje proces ładowania oraz zabezpiecza akumulator przed nadmiernym rozładowaniem i zwarcie, co znacząco podnosi niezawodność całego urządzenia. Dzięki takiemu rozwiązaniu sorter po zaprogramowaniu i podłączeniu do sieci może realizować procesy segregacji w sposób ciągły, będąc jednocześnie odpornym na chwilowe zakłócenia energii [6].



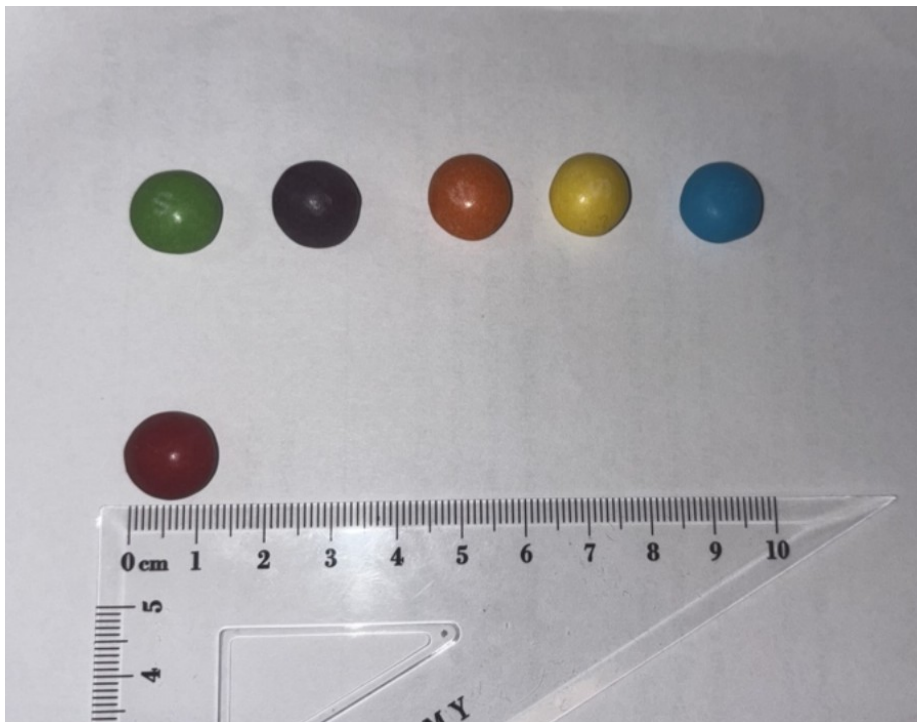
*Rys. 2.5.1. Moduł impulsowej przetwornicy DC-DC z funkcją UPS i zintegrowanym systemem ochrony ogniwo litowych,
Źródło: Dokumentacja techniczna modułu.*

3. OPRACOWANIE KONCEPCJI ORAZ PROJEKTU MECHANICZNEGO I ELEKTRONICZNEGO URZĄDZENIA SORTUJĄCEGO

3.1. Założenia projektowe i dobór surowca sortowanego

Głównym celem projektu było zbudowanie autonomicznego urządzenia, którego zadaniem jest rozpoznawanie kolorów obiektów w czasie rzeczywistym i ich precyzyjna segregacja. Na etapie planowania zdecydowano, że maszyna będzie budowana jako dedykowany prototyp, zoptymalizowany pod kątem konkretnego surowca testowego. Wybór padł na drażetki „Skittles”, co było kluczowe dla zapewnienia niezawodności mechanicznej całego systemu. Zastosowanie konkretnego modelu surowca pozwoliło na bardzo dokładne dopasowanie wymiarów wszystkich części ruchomych już na etapie koncepcyjnym. Dzięki temu, że drażetki te mają powtarzalny kształt i rozmiar, możliwe było zminimalizowanie luzów technicznych, co w prototypach tego typu jest kluczowe, aby uniknąć blokowania się elementów w kanałach transportowych.

Ważnym aspektem było również zapewnienie szerokiej gamy barw do przetestowania algorytmu. Podstawowy zestaw kolorów (czerwony, pomarańczowy, żółty, zielony, fioletowy) rozszerzono o drażetki z edycji „Tropical”, aby wprowadzić do testów kolor niebieski. Dzięki takiemu zróżnicowaniu możliwe było przeprowadzenie pełnej, sześciopiętowej kalibracji czujnika TCS34725. Pozwoliło to na precyzyjne wyznaczenie progów dla składowych RGB w kodzie sterującym, co bezpośrednio przekłada się na skuteczność sortowania. Dokładne pomiary suwmiarką wykazały, że średnia średnica obiektu wynosi 13 mm, a jego wysokość to 9 mm. Uznano te wartości za parametry wejściowe, do których w dalszej kolejności dostosowano geometrię wszystkich wewnętrznych kanałów przelotowych oraz elementów mechanicznych odpowiedzialnych za ruch cukierka.



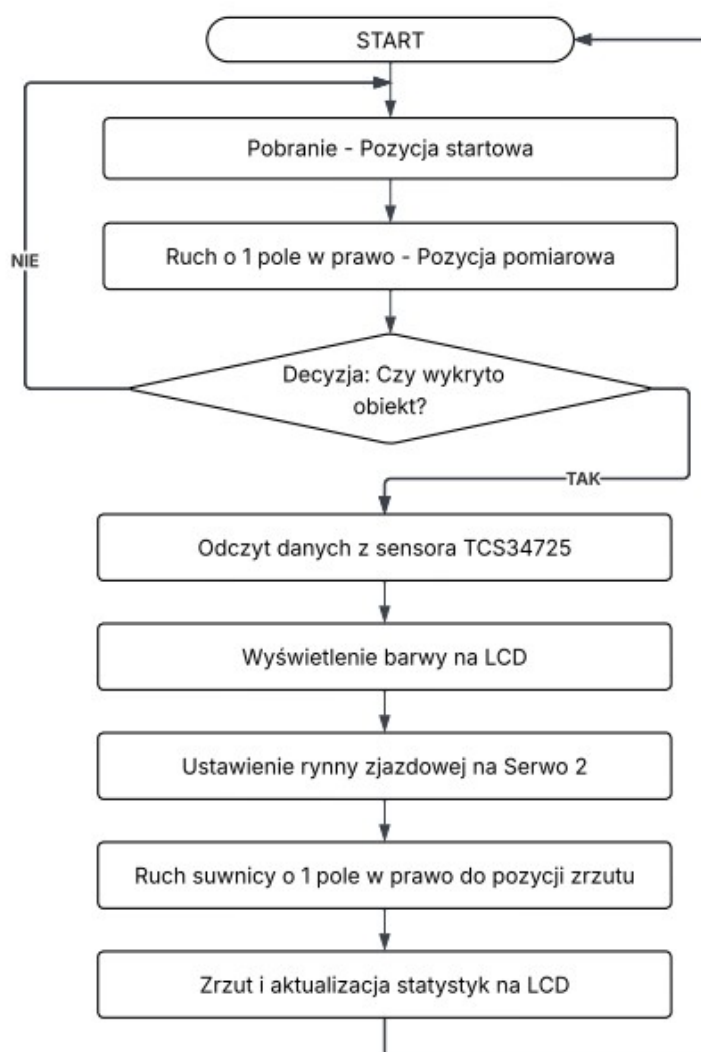
*Rys. 3.1.1. Zestawienie kolorystyczne drażetek, stanowiące pełne spektrum barw podlegających procesowi sortowania.
Opracowanie własne.*

3.2. Koncepcja działania i logika procesu

Po dokładnym określeniu wymiarów drażetek, przystąpiono do opracowania koncepcji działania całego urządzenia. Głównym założeniem było stworzenie systemu, który będzie w stanie odizolować pojedynczą sztukę surowca z zasobnika i przetransportować ją pod czujnik bez ryzyka zablokowania mechanizmu. Zdecydowano się na konstrukcję wielopoziomową, co pozwoliło na maksymalne wykorzystanie siły grawitacji przy przemieszczaniu obiektów między poszczególnymi etapami procesu.

Proces rozpoczyna się na najwyższym poziomie, gdzie znajduje się zasobnik na surowiec. Zastosowano w nim pionowy kanał o szerokości dostosowanej do średnicy drażetki. Takie rozwiązanie wymusza na obiektach ustawienie się w rzędzie jeden pod drugim, co sprawia, że pod wpływem własnego ciężaru wpadają one pojedynczo do mechanizmu dozującego znajdującego się na niższym piętrze. Mechanizm ten może zostać zrealizowany w formie suwnicy poruszającej się po półokręgu. Jej zadaniem jest odebranie jednej drażetki z pionowej kolejki i przetransportowanie jej w poziomie do kolejnych punktów technologicznych.

Rozdzielenie procesu na etapy (pobranie, pomiar, zrzut) pozwoli wyeliminować błędy odczytu. Dzięki temu, że suwnica zatrzymuje się bezpośrednio pod sensorem, obiekt pozostaje nieruchomy przez czas potrzebny na precyzyjną analizę barwy. Cała logika sterowania, uwzględniająca ruchy mechaniczne oraz procesy decyzyjne mikrokontrolera, została przedstawiona na schemacie blokowym (Rys. 3.2.1).



Rys. 3.2.1. Schemat blokowy algorytmu sterującego urządzeniem z uwzględnieniem pętli detekcji obecności surowca oraz sekwencyjnej pracy serwomechanizmów. Opracowanie własne.

Działanie urządzenia opisane na schemacie (Rys. 3.2.1) opiera się na cyklicznie powtarzanej pętli programowej. Cykl zaczyna się od ustawienia suwnicy w pozycji startowej, bezpośrednio pod kanałem zasobnika. Po pobraniu drażetki, serwomechanizm przesuwa suwnicę o jedną pozycję w prawo do komory pomiarowej.

W tym punkcie algorytm realizuje kluczową funkcję zabezpieczającą - weryfikację obecności obiektu w gnieździe. Jeśli czujnik nie wykryje drażetki (co może się zdarzyć przy chwilowym zablokowaniu surowca w kanale górnym), program przerywa standardowy tok pracy. Zamiast przechodzić do analizy koloru, suwnica jest cofana do pozycji startowej w celu ponownej próby pobrania elementu. Takie rozwiązanie sprawia, że urządzenie jest odporne na błędy mechaniczne i nie wykonuje „pustych” cykli sortowania.

W przypadku wykrycia obiektu, mikrokontroler pobiera dane z czujnika TCS34725 i dokonuje ich klasyfikacji do jednej z sześciu zaprogramowanych barw. Po zidentyfikowaniu koloru, nazwa barwy jest wysyłana do sterownika LCD. Jednocześnie drugi serwomechanizm ustawia zjeżdżalnię wyjściową pod odpowiednim kątem, kierując ją do właściwego pojemnika. Gdy rynna jest już gotowa, suwnica wykonuje kolejny ruch w prawo, ustawiając się nad otworem zrzutowym. Drażetka opada grawitacyjnie na zjeżdżalnię i trafia do celu. Ostatnim etapem jest aktualizacja liczników na wyświetlaczu oraz powrót suwnicy o dwie pozycje w lewo, co przywraca układ do stanu początkowego i pozwala na rozpoczęcie kolejnego cyklu.

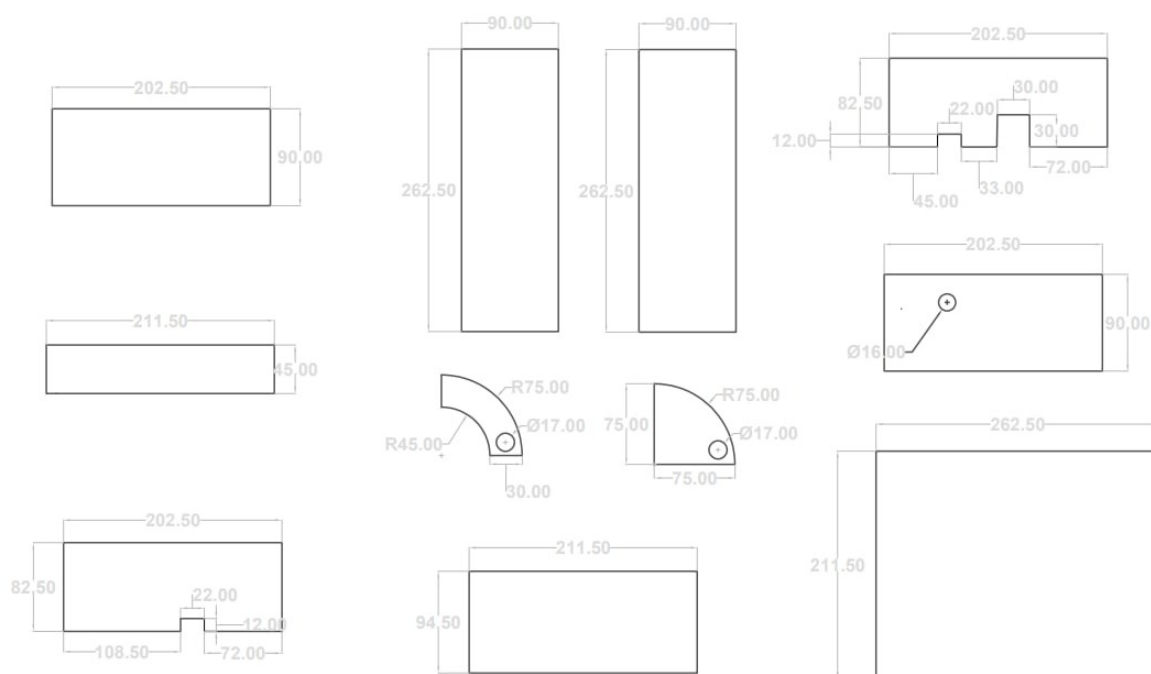
3.3. Projektowanie mechaniczne w środowisku AutoCAD

Projektowanie techniczne w programie AutoCAD zostało poprzedzone etapem budowy prowizorycznego modelu z kartonu. Celem tego działania było praktyczne sprawdzenie, czy założona koncepcja „piętrowa” i wykorzystanie grawitacji faktycznie sprawdzą się przy tak małych i lekkich obiektach, jakimi są cukierki Skittles. Testy na kartonowym prototypie pozwoliły zaobserwować, jak cukierki przemieszczają się między poziomami i czy stawiają opory podczas ruchu. Dzięki temu udało się uniknąć błędów konstrukcyjnych jeszcze przed przystąpieniem do rysowania ostatecznych elementów.



Rys. 3.3.1. Prototyp funkcjonalny wykonany w celu weryfikacji założeń mechanicznych oraz sprawdzenia geometrii zsuwni grawitacyjnej przed etapem modelowania docelowego. Opracowanie własne.

Głównym zadaniem w programie AutoCAD było precyzyjne rozplanowanie „domku” tak, aby wszystkie poziomy i otwory przelotowe idealnie się ze sobą pokrywały. W konstrukcji zdecydowano się na wykorzystanie gotowej rurki o średnicy wewnętrznej 15 mm jako głównego kanału doprowadzającego, która w testach praktycznych najlepiej sprawdziła się w roli przewodnicy. Z kolei gniazdo w samym suwaku zaprojektowano ze znacznie większym luzem, nadając mu średnicę 17 mm. Taki zabieg miał na celu ułatwienie „wpadania” cukierka do środka oraz zapewnienie swobodnego przesuwu suwnicy bez ryzyka, że krawędź drażetki zahaczy o obudowę podczas ruchu poziomego.

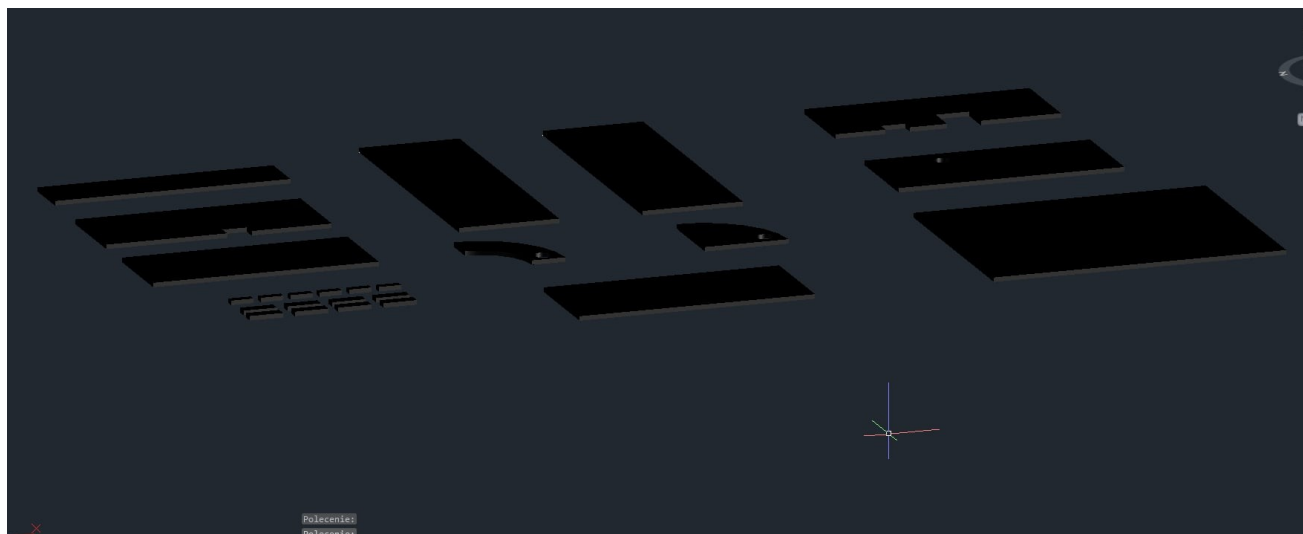


Rys 3.3.2 Projekt techniczny elementów konstrukcyjnych wykonany w środowisku AutoCAD. Opracowanie własne.

Po pomyślnych testach na prototypie z kartonu oraz zaprojektowaniu wymiarów przystąpiono do projektowania docelowego urządzenia w programie AutoCAD. Na tym etapie wcześniejsze szkice 2D zmieniono w precyzyjne modele trójwymiarowe. Pozwoliło to na dokładne zaplanowanie miejsc, w których miały zostać zamontowane serwomechanizmy MG-90S oraz czujnik koloru TCS34725. Dzięki temu wszystkie elementy pasowały do siebie idealnie, co było kluczowe dla stabilnej pracy całego mechanizmu.

Bardzo ważną częścią projektu było stworzenie specjalnej komory pomiarowej. Została ona zaprojektowana tak, aby była ciasna i szczelna. Takie rozwiązanie jest najlepsze dla czujnika, ponieważ mała przestrzeń ogranicza rozproszone światło i sprawia, że sensor znajduje się zawsze w tej samej, niewielkiej odległości od cukierka. Dzięki temu pomiar koloru jest stabilny i powtarzalny za każdym razem.

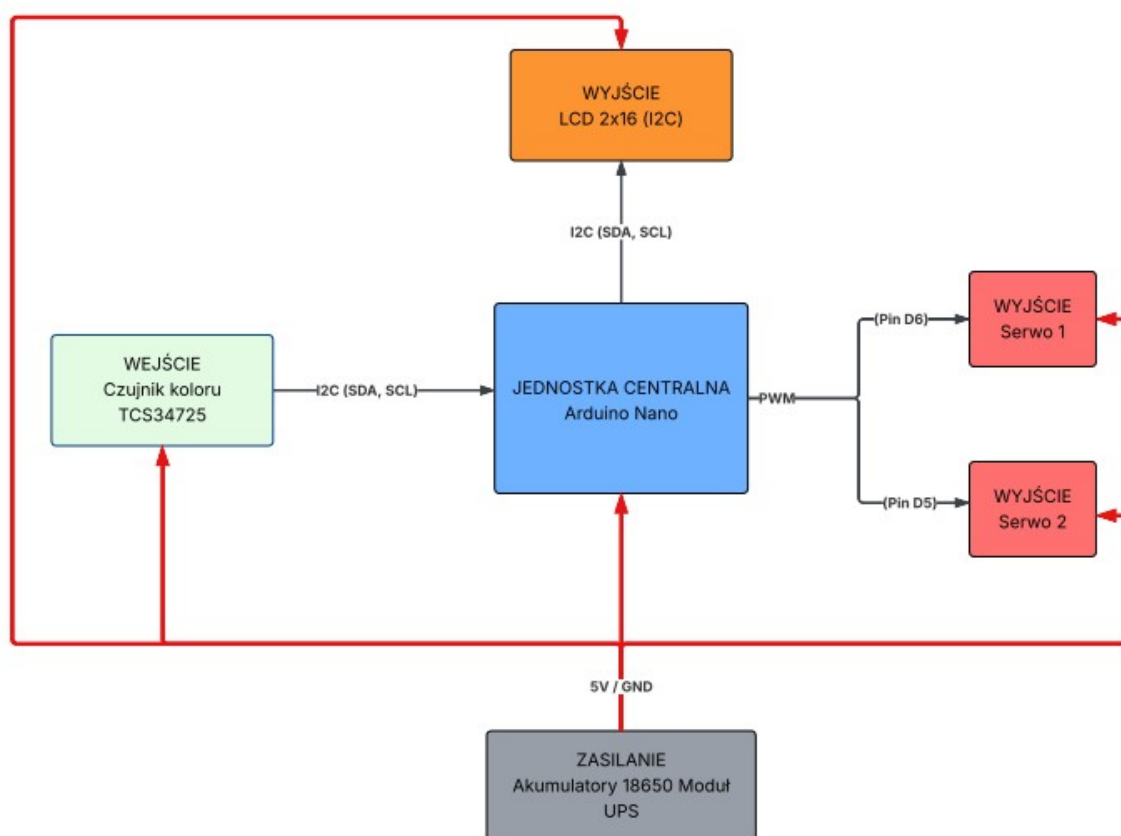
Aby całkowicie odciąć czujnik od światła z zewnątrz, zdecydowano się na druk 3D z ciemnego, matowego materiału. Wybór ten nie był przypadkowy - ciemny kolor blokuje światło z pokoju, które mogłoby oszukać czujnik, a matowa powierzchnia sprawia, że światło z diody czujnika nie odbija się od ścianek wewnątrz obudowy. Dzięki temu sensor „widzi” tylko kolor cukierka, a nie refleksy od obudowy, co znacznie poprawia dokładność rozpoznawania barw. Samodzielny montaż tych elementów umożliwił precyzyjne ustawienie podajnika względem rynny, którą cukierki zsuwają się do pojemników. Takiej dokładności nie udałooby się osiągnąć, budując urządzenie ręcznie z przypadkowych materiałów.



Rys. 3.3.3. Zrzut ekranu z programu AutoCAD przedstawiający trójwymiarowe modele części składowych przygotowane do procesu druku 3D. Opracowanie własne.

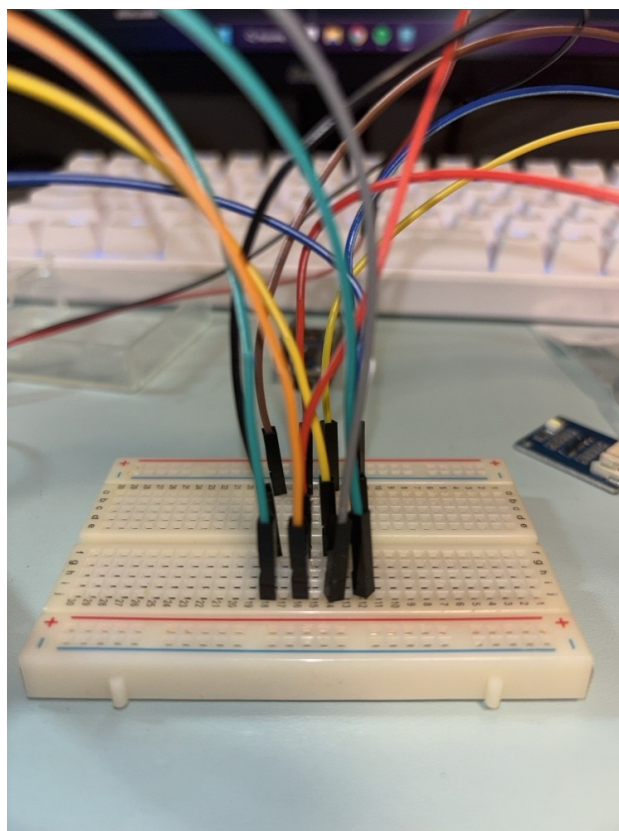
3.4. Projekt układu elektronicznego w środowisku KiCad

Projektowanie warstwy sprzętowej urządzenia rozpoczęto od zdefiniowania fundamentów logicznych systemu, co pozwoliło na precyzyjne określenie interakcji zachodzących między poszczególnymi podzespołami. Przed przystąpieniem do prac nad fizyczną strukturą obwodu drukowanego, opracowano szczegółowy schemat architektury elektronicznej (Rys. 3.4.1). Dokumentacja ta obrazuje hierarchię komunikacyjną urządzenia, w której jednostka centralna - mikrokontroler Arduino Nano - pełni rolę nadrzędną, zarządzając przepływem danych z czujnika koloru TCS34725 poprzez magistralę I2C oraz sterując układami wykonawczymi. Schemat ten stał się mapą drogową dla dalszych prac projektowych, definiując sposób dystrybucji zasilania z modułu UPS oraz integrację interfejsu użytkownika w postaci wyświetlacza LCD.

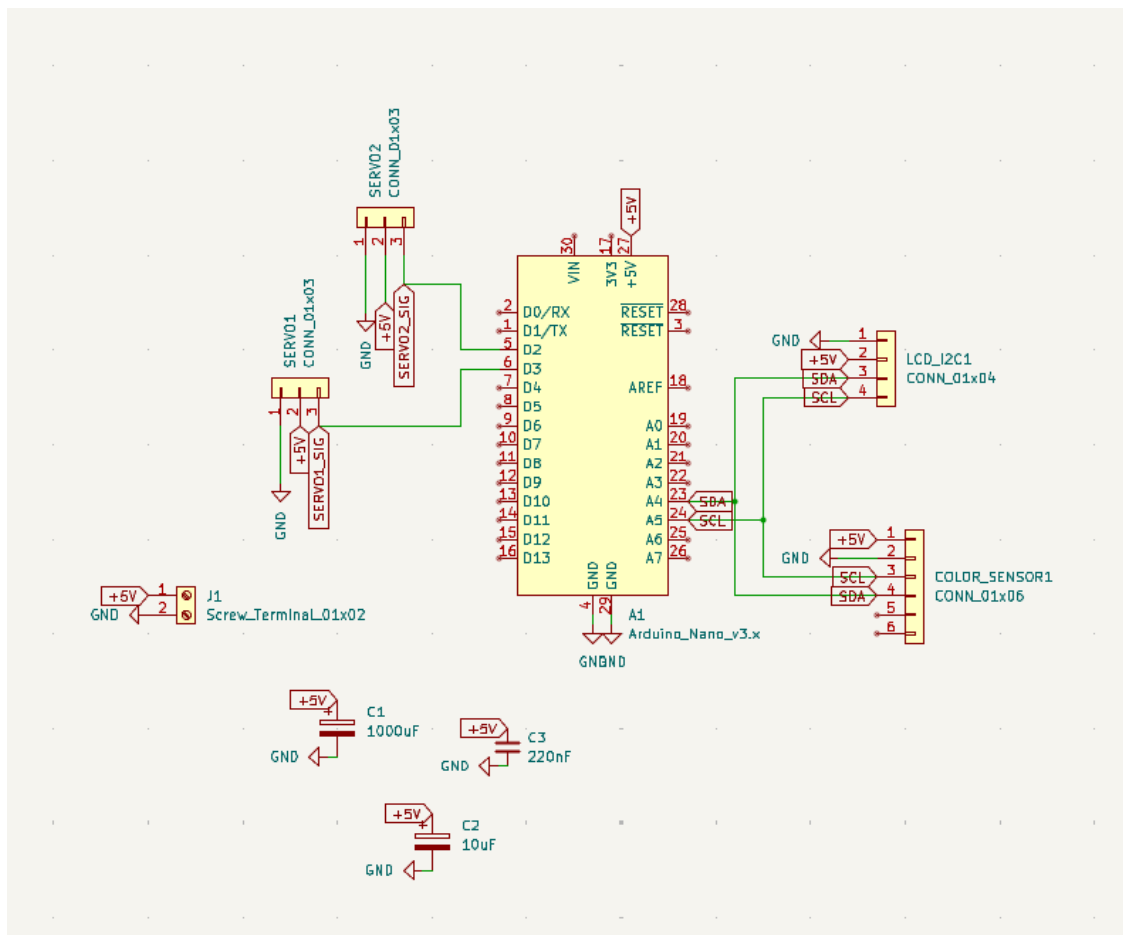


Rys. 3.4.1. Schemat blokowy architektury elektronicznej systemu. Opracowanie własne.

Po zaplanowaniu teorii, przystąpiono do budowy prototypu. Wykorzystano do tego płytkę stykową (Rys. 3.4.2.), która pozwoliła na szybkie łączenie elementów bez konieczności lutowania. Ten etap był niezbędny, aby w praktyce sprawdzić, czy wszystkie połączenia są poprawne, a program sterujący urządzeniem działa zgodnie z założeniami. Testy potwierdziły, że czujnik prawidłowo rozpoznaje kolory, a program sprawnie steruje ruchami silników. Mimo to, prototyp na płytce stykowej był tylko rozwiązaniem tymczasowym. Luźne przewody mogły łatwo wypaść, co powodowało błędy w działaniu urządzenia. Dodatkowo, takie połączenia są podatne na zakłócenia elektryczne. Aby urządzenie było trwałe, niezawodne i zajmowało mniej miejsca, zdecydowano się na zaprojektowanie własnej płytki drukowanej (PCB) w programie KiCad. Było to kluczowe, ponieważ sorter posiada ruchome części. Drgania powstające podczas pracy silników mogłyby łatwo rozłączyć zwykłe kable, natomiast polutowana płytka PCB gwarantuje, że wszystkie połączenia pozostaną stabilne i bezpieczne.



Rys. 3.4.2. Wstępny prototyp układu sterowania wykonany na płytce stykowej. Opracowanie własne.

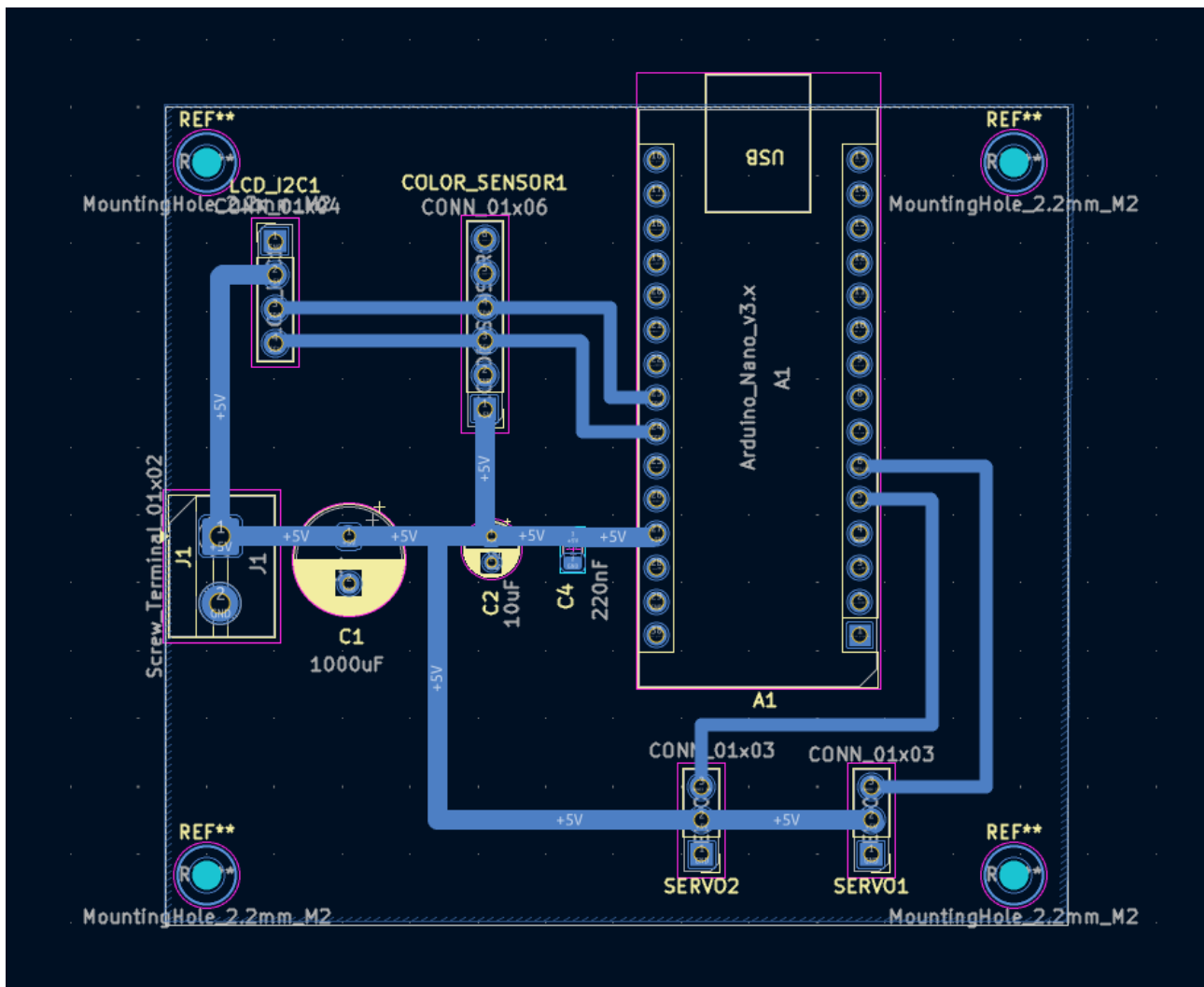


Rys. 3.4.3. Schemat ideowy układu w programie KiCad z widocznymi kondensatorami i złączami modułów.
Opracowanie własne.

Głównym założeniem podczas projektowania PCB była optymalna integracja wszystkich modułów w ramach jednej kompaktowej platformy. Na płytce rozmieszczono dedykowane gniazda pinowe typu goldpin, które umożliwiają łatwy i pewny montaż kluczowych podzespołów. Sercem układu jest gniazdo dla mikrokontrolera Arduino Nano (A1), wokół którego zaprojektowano wejścia dla pozostałych modułów. Dla komunikacji wizualnej przewidziano czteropinowe złącze LCD_I2C1 (standard CONN_01x04), udostępniające linie zasilania oraz magistralę danych I2C (SDA, SCL). W bezpośrednim sąsiedztwie zlokalizowano sześciopinowe złącze COLOR_SENSOR1 (CONN_01x06), dedykowane dla czujnika TCS34725. Zastosowanie złączy o określonym rastrze i liczbie pinów pozwoliło na uporządkowanie struktury przewodów i wyeliminowanie ryzyka błędnego podpięcia sensorów.

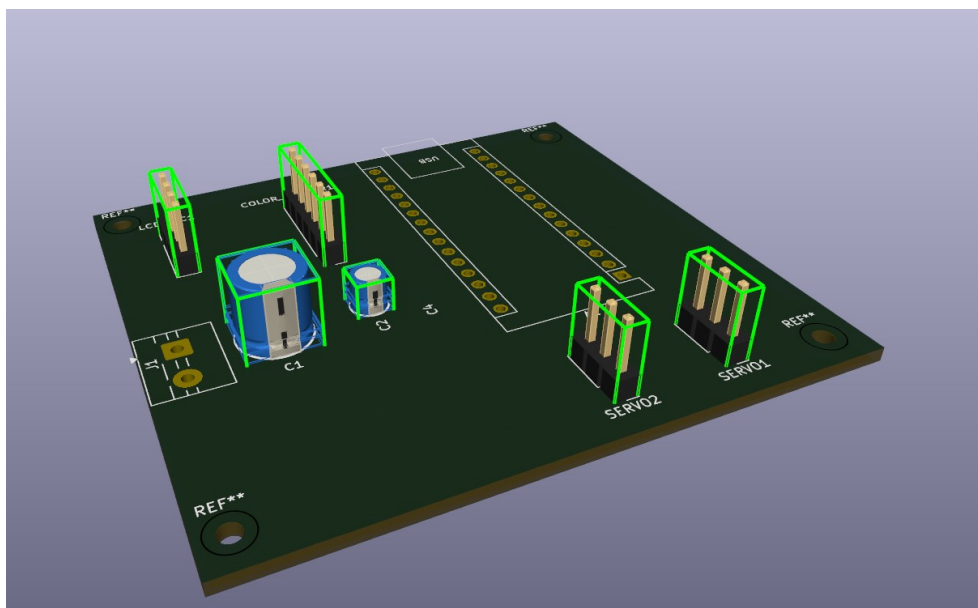
W dolnej sekcji płytki zaprojektowano dwa niezależne, trzypinowe gniazda SERVO1 oraz SERVO2 (CONN_01x03), doprowadzone do pinów cyfrowych mikrokontrolera, co zapewnia precyzyjne sterowanie ramieniem dozownika i rynną zsuwniczą. Kluczowym wyzwaniem inżynierskim na tym etapie było zapewnienie stabilności zasilania, ponieważ serwomechanizmy w momencie startu generują znaczne piki prądowe, które w fazie prototypu powodowały niestabilność pracy procesora. Aby temu zaradzić, w projekcie zaimplementowano rozbudowaną sekcję filtracji zasilania, składającą się z kaskady kondensatorów o różnych parametrach fizycznych.

Zastosowano kondensator elektrolityczny o dużej pojemności 1000 μF (C1), który pełni rolę bufora energii i magazynuje ładunek niezbędny do pokrycia gwałtownego zapotrzebowania na prąd przez silniki serw. Obok niego, w celu wielostopniowego wygładzania napięcia, umieszczono kondensator elektrolityczny 10 μF (C2) oraz kondensator ceramiczny o pojemności 220 nF (C3). Ten ostatni element jest szczególnie istotny, ponieważ dzięki swojej konstrukcji skutecznie eliminuje zakłócenia o wysokich częstotliwościach oraz szumy impulsowe, które mogłyby negatywnie wpływać na precyzję odczytów z czujnika koloru lub zakłócać komunikację na magistrali I2C.

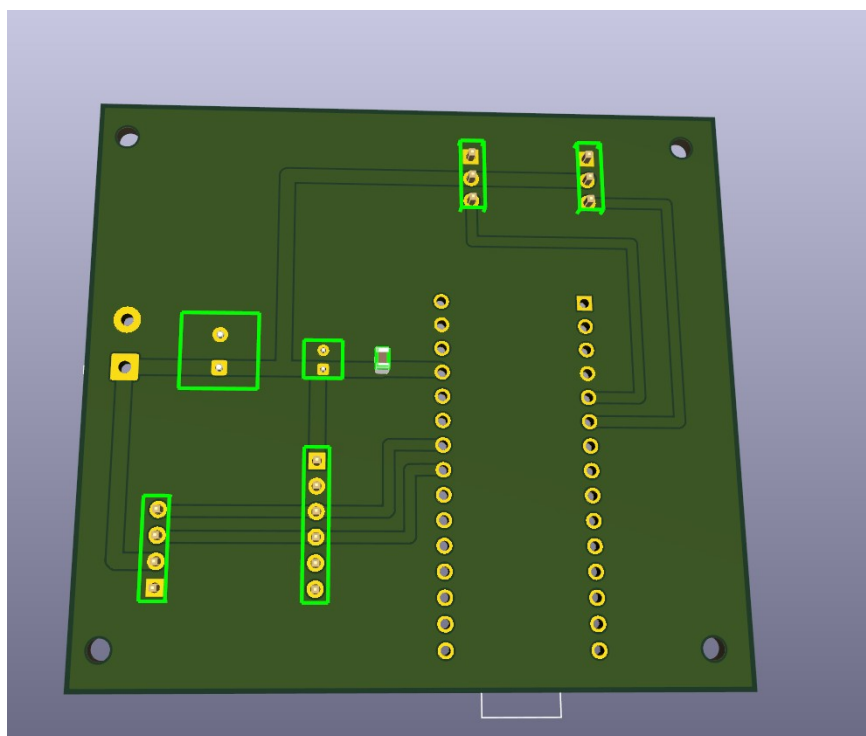


Rys. 3.4.4. Projekt ścieżek na warstwie dolnej PCB (B.Cu). Opracowanie własne.

Podczas projektowania samej logiki ścieżek, szczególną uwagę zwrócono na separację torów prądowych oraz zastosowanie odpowiednich szerokości ścieżek dla linii zasilających, doprowadzonych do głównego złącza śrubowego J1. Projekt został uzupełniony o cztery otwory montażowe rozmieszczone w narożnikach płytki, co umożliwia stabilne i pewne osadzenie. Całość prac w programie KiCad zwieńczono wygenerowaniem wizualizacji 3D, co pozwoliło na ostateczną weryfikację poprawności rozmieszczenia komponentów względem elementów konstrukcyjnych zsuwni i dozownika przed finalną realizacją układu.



Rys 3.4.5 Wizualizacja 3D górnej warstwy płytki z obsadzonymi komponentami. Opracowanie własne



Rys. 3.4.6. Wizualizacja 3D dolnej warstwy płytki przedstawiająca prowadzenie ścieżek. Opracowanie własne.

4. IMPLEMENTACJA UKŁADU STERUJĄCEGO I OPROGRAMOWANIA REALIZUJĄCEGO PROCES SORTOWANIA OBIEKTÓW WEDŁUG BARWY

4.1. Budowa obudowy układu

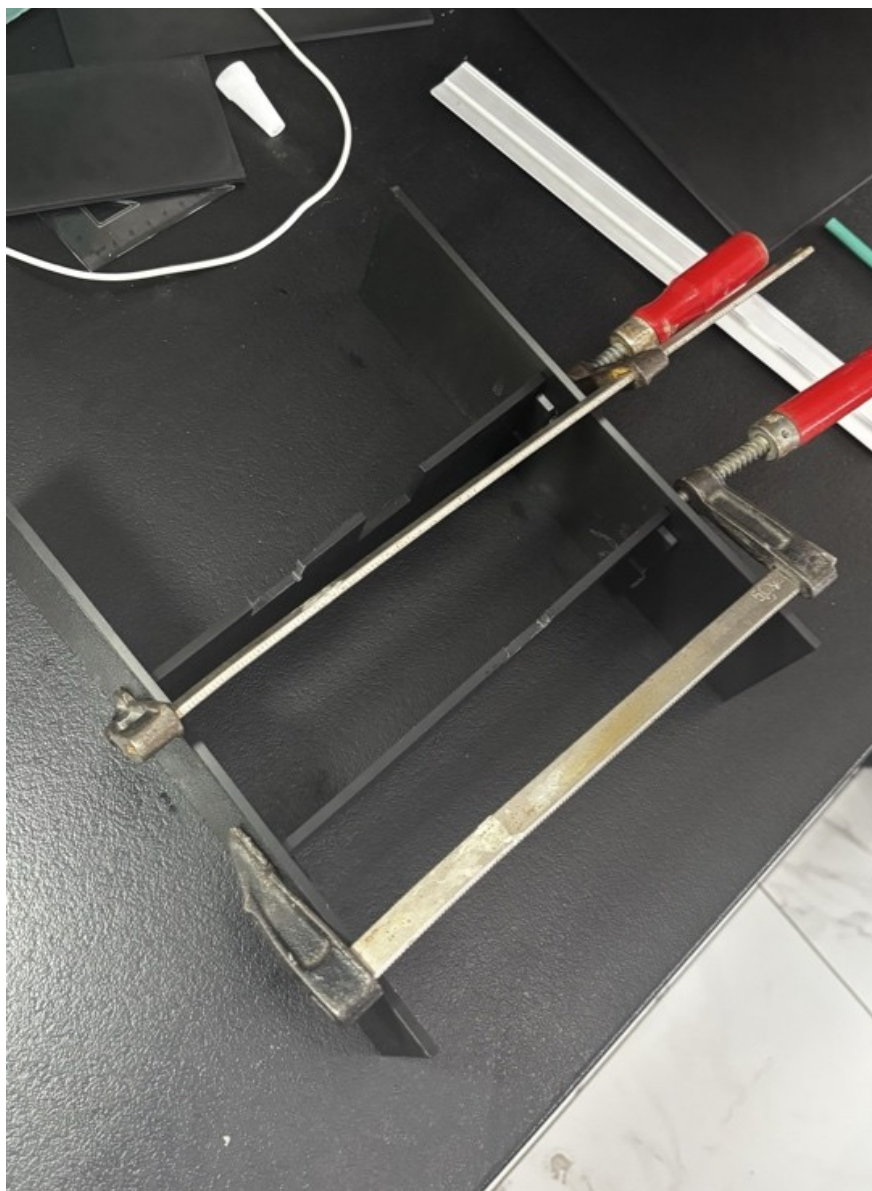
Realizację fizyczną urządzenia rozpoczęto od przygotowania elementów uzyskanych w procesie druku 3D (Rys. 4.4.1). Dzięki przemyślanemu zorientowaniu modeli na stole roboczym drukarki, udało się uzyskać czyste i gładkie wydruki bez konieczności generowania dodatkowych struktur wspierających. Wyeliminowanie podpór nie tylko skróciło czas produkcji, ale również pozwoliło na zachowanie wysokiej jakości powierzchni bez śladów mechanicznego usuwania materiału pomocniczego. Na tym etapie przeprowadzono dokładną weryfikację jakości wszystkich płaskich elementów, sprawdzając ich wymiary oraz płaszczyzny przylegania pod kątem ewentualnych odkształceń.

Kluczowym etapem prac było precyzyjne pasowanie elementów. Ze względu na naturalny skurcz materiału termoplastycznego, niektóre otwory montażowe oraz krawędzie stykowe wymagały delikatnej obróbki mechanicznej przy użyciu pilników i papieru ściernego o drobnej ziarnistości. Proces ten pozwolił na idealne wygładzenie zsuwni oraz kanałów prowadzących, co było niezbędne dla zapewnienia swobodnego ruchu cukierków i wyeliminowania ryzyka ich blokowania się wewnątrz mechanizmu.



Rys. 4.1.1. Zestaw płaskich elementów obudowy po zakończeniu procesu druku 3D.

Następnie przystąpiono do głównego etapu spajania konstrukcji za pomocą kleju akrylowego. Podczas klejenia priorytetem było zachowanie idealnej osiowości oraz prostopadłości ścianek, aby suwak dozujący mógł poruszać się płynnie i bez zbędnych oporów mechanicznych. W tym celu wykorzystano ściski stolarskie (Rys. 4.1.2.), które pozwoliły na stabilne unieruchomienie ścianek bocznych względem podstawy na czas wiązania spoiny. Zastosowanie narzędzi dociskowych zapobiegło przesunięciom elementów, co zagwarantowało sztywność i stabilność całej wieży sortera.



Rys. 4.1.2. Proces montażu i klejenia korpusu urządzenia z wykorzystaniem ścisków stolarskich.

Równoległe z montażem korpusu przystąpiono do instalacji kluczowych podzespołów. Silniki MG-90S osadzono w dedykowanych gniazdach przy użyciu śrub montażowych, weryfikując ręcznie, czy ramiona dozownika i rynny mają pełny zakres obrotu i nie kolidują ze ściankami obudowy. Niezwykle istotnym zadaniem było precyzyjne umieszczenie czujnika koloru bezpośrednio nad szyną przesuwną. Sensor został zamontowany w taki sposób, aby dokonywać pomiaru barwy w momencie, gdy cukierek zostaje unieruchomiony przez specjalnie zaprojektowaną ściankę oporową. Ścianka ta, umieszczona w ściśle określonej odległości od

czujnika, zapewnia stały punkt pomiarowy dla każdego obiektu. Takie rozwiązanie gwarantuje powtarzalność wyników i optymalne warunki oświetleniowe dla sensora, co bezpośrednio przekłada się na wysoką skuteczność procesu segregacji barwnej.

W górnej części konstrukcji zamontowano układ doprowadzający surowiec, składający się z lejka oraz rurki prowadzącej. Jako kanał transportowy wykorzystano gotową rurkę o promieniu 15 mm, którą docięto na długość około 5 cm. Została ona trwale połączona z lejkiem, tworząc pionowy podajnik grawitacyjny. Takie rozwiązanie techniczne wymusza na cukierkach spadających z lejka odpowiednie ustawienie się - dzięki ograniczonej średnicy rurki obiekty układają się jeden po drugim, co jest kluczowe dla poprawnej pracy suwaka dozującego, który musi pobierać dokładnie jeden cukierek w każdym cyklu pracy.

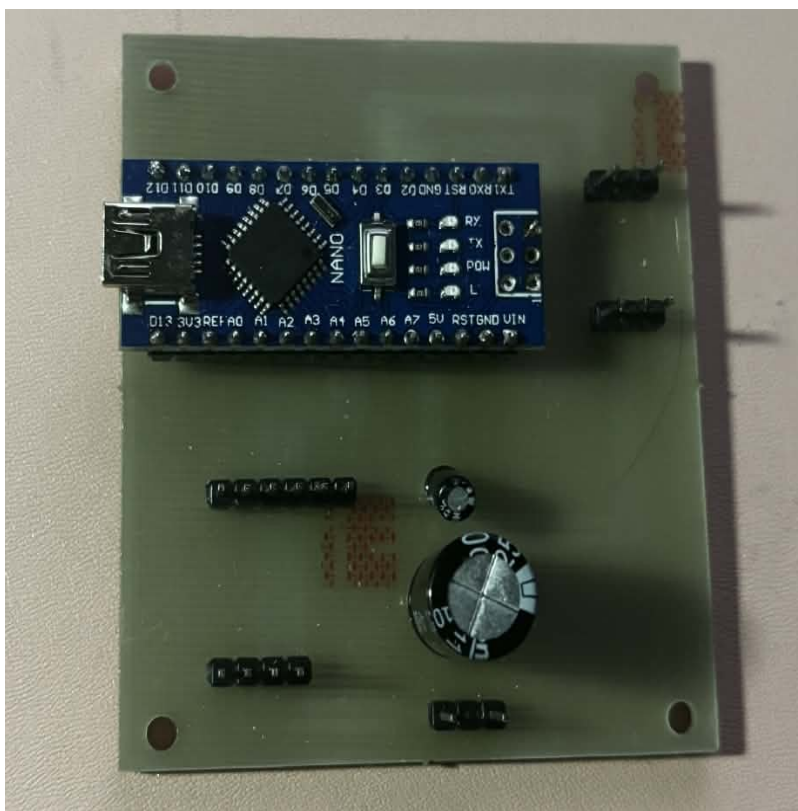
Bezpośrednio pod suwakiem dozującym umieszczono rynnę transportową, połączoną z ramieniem drugiego serwomechanizmu. Element ten zaprojektowano w celu zapewnienia sprawnego przemieszczenia obiektu z punktu dozowania do dalszych sekcji urządzenia. Powierzchnia robocza rynny została pokryta specjalistyczną, czarną taśmą o wysokim stopniu gładkości. Zastosowanie takiej powłoki ma na celu radykalne zmniejszenie współczynnika tarcia między sortowanym elementem a powierzchnią zjeżdżalni. Dzięki gładkiej strukturze taśmy, minimalizowane jest ryzyko klinowania się obiektów, co gwarantuje powtarzalną prędkość ich przemieszczania się oraz płynność pracy całego mechanizmu grawitacyjnego.

Po przygotowaniu struktury mechanicznej przystąpiono do montażu warstwy elektronicznej, opierając się na zaprojektowanej wcześniej płytce drukowanej. Proces rozpoczęto od obsadzenia i przylutowania gniazd typu goldpin pod mikrokontroler Arduino Nano oraz moduły peryferyjne, co pozwala na ich bezinwazyjny demontaż w celach serwisowych. Podczas lutowania szczególną uwagę poświęcono sekcji filtracji zasilania, montując kaskadę kondensatorów (1000 μ F, 10 μ F oraz 220 nF) zgodnie z polaryzacją wyznaczoną na warstwie opisowej płytki. Solidne połączenie lutowane dużego kondensatora elektrolitycznego było kluczowe, gdyż stanowi on główny bufor energii zapobiegający spadkom napięcia podczas nagłych skoków poboru prądu przez serwomechanizmy.

Kolejnym etapem było wykonanie okablowania łączącego płytkę sterującą z elementami wykonawczymi zamontowanymi w obudowie. Przewody od serwomechanizmów oraz czujnika koloru zostały zakończone odpowiednimi wtykami i wpięte w dedykowane gniazda na PCB. Dzięki zastosowaniu złącza śrubowego J1, możliwe było pewne doprowadzenie zasilania głównego. Wyświetlacz LCD został połączony za pomocą czteroprzewodowej magistrali I2C, co pozwoliło na

zachowanie estetyki i porządku wewnątrz obudowy, unikając plątaniny kabli występującej w fazie prototypu.

Ostatnim, najważniejszym krokiem było zainstalowanie systemu zasilania autonomicznego. Wykorzystano do tego moduł UPS zintegrowany z koszykiem na dwa ogniwa litowo-jonowe typu 18650. Do zasilania układu wybrano wysokoprądowe ogniwa Samsung INR18650-25R o pojemności 2500 mAh każda. Zastosowanie tych konkretnych akumulatorów pozwoliło na uzyskanie prądu rozładowania o natężeniu do 20 A, co z dużym zapasem pokrywa zapotrzebowanie prądowe dwóch serwomechanizmów w momencie ich jednoczesnego startu. W celu zwiększenia łatwości obsługi urządzenia, wejście J1 na płytce dodatkowo połączono z przyciskiem bistabilnym, który został zamontowany w górnej części zamykającej obudowę. Przycisk ten zintegrowano z układem podtrzymania zasilania, co zapewnia stabilność pracy systemu i chroni przed skutkami nagłych przerw w dostawie energii.



Rys. 4.1.3. Gotowa płytka sterująca PCB z przyłutowanymi gniazdami modułów oraz zestawem kondensatorów filtrujących zasilanie.

4.2. Program do kalibracji i analizy surowych danych

Zanim powstał ostateczny algorytm, przygotowano specjalny program kalibracyjny. Został on opracowany w zintegrowanym środowisku programistycznym Arduino IDE (ang. *Integrated Development Environment*), które jest standardowym narzędziem do tworzenia oprogramowania dla mikrokontrolerów z rodziny Arduino. Kod źródłowy napisano w języku C++, wzbogaconym o dedykowane biblioteki sprzętowe, co pozwoliło na zarządzanie rejestrami czujnika. Jego jedynym zadaniem był ciągły odczyt wartości Red (R), Green (G), Blue (B) oraz Clear (C) i przesyłanie ich do komputera. Dzięki temu, wkładając pod czujnik różne cukierki, można było zaobserwować, jak zmieniają się liczby w zależności od barwy.

Listing 4.2.1. Program do odczytu surowych danych kolorometrycznych

```
1 #include <Wire.h>
2 #include "Adafruit_TCS34725.h"
3
4
5 Adafruit_TCS34725 tcs = Adafruit_TCS34725(TCS34725_INTEGRATIONTIME_50MS,
6     TCS34725_GAIN_4X);
7
8 void setup() {
9     Serial.begin(9600);
10
11     if (tcs.begin()) {
12         Serial.println("Czujnik TCS34725 znaleziony!");
13     } else {
14         Serial.println("Brak czujnika... sprawdź połączenia.");
15         while (1);
16     }
17 }
18
19 void loop() {
20     uint16_t r, g, b, c;
21
22     tcs.getRawData(&r, &g, &b, &c);
23
24     Serial.print("R: "); Serial.print(r); Serial.print(" ");
25     Serial.print("G: "); Serial.print(g); Serial.print(" ");
26     Serial.print("B: "); Serial.print(b); Serial.print(" ");
27     Serial.print("Clear: "); Serial.print(c);
28     Serial.println();
29
30     delay(200);
31 }
32 }
```

Przedstawiony algorytm kalibracyjny realizuje sekwencyjny odczyt danych z rejestrów czujnika TCS34725 i ich transmisję do komputera PC. W pierwszej kolejności program dokonuje inicjalizacji komunikacji szeregowej z prędkością 9600 bodów oraz sprawdza poprawność połączenia z układem sensorycznym poprzez wywołanie metody `begin()`. W przypadku poprawnej detekcji czujnika, program przechodzi do pętli głównej, w której definiowane są cztery zmienne typu `uint16_t` przeznaczone do przechowywania surowych danych o składowych widmowych.

Kluczową operacją jest wykorzystanie funkcji `getRawData`, która pobiera bezpośrednio wartości z 16-bitowych przetworników analogowo-cyfrowych dla kanałów czerwonego, zielonego, niebieskiego oraz światła białego. Dzięki zastosowaniu czasu integracji wynoszącego 50 ms oraz czterokrotnego wzmocnienia sygnału (`GAIN 4X`), uzyskano optymalny balans między szybkością odświeżania danych a ich czułością, co pozwoliło na stabilną obserwację zmian parametrów liczbowych w zależności od barwy badanego obiektu. Dane te są następnie formatowane i wysyłane do monitora portu szeregowego przy użyciu instrukcji `Serial.print`.

Wykorzystanie biblioteki `Wire.h` umożliwiło stabilną komunikację poprzez magistralę I2C, natomiast biblioteka `Adafruit_TCS34725.h` dostarczyła gotowych metod do obsługi układu, co znacząco uprościło strukturę kodu. Tak przygotowane oprogramowanie pozwoliło na precyzyjne wyznaczenie progów decyzyjnych dla każdego koloru, uwzględniając jednocześnie specyficzny stan braku obiektu w komorze pomiarowej.

4.3. Analiza danych pomiarowych

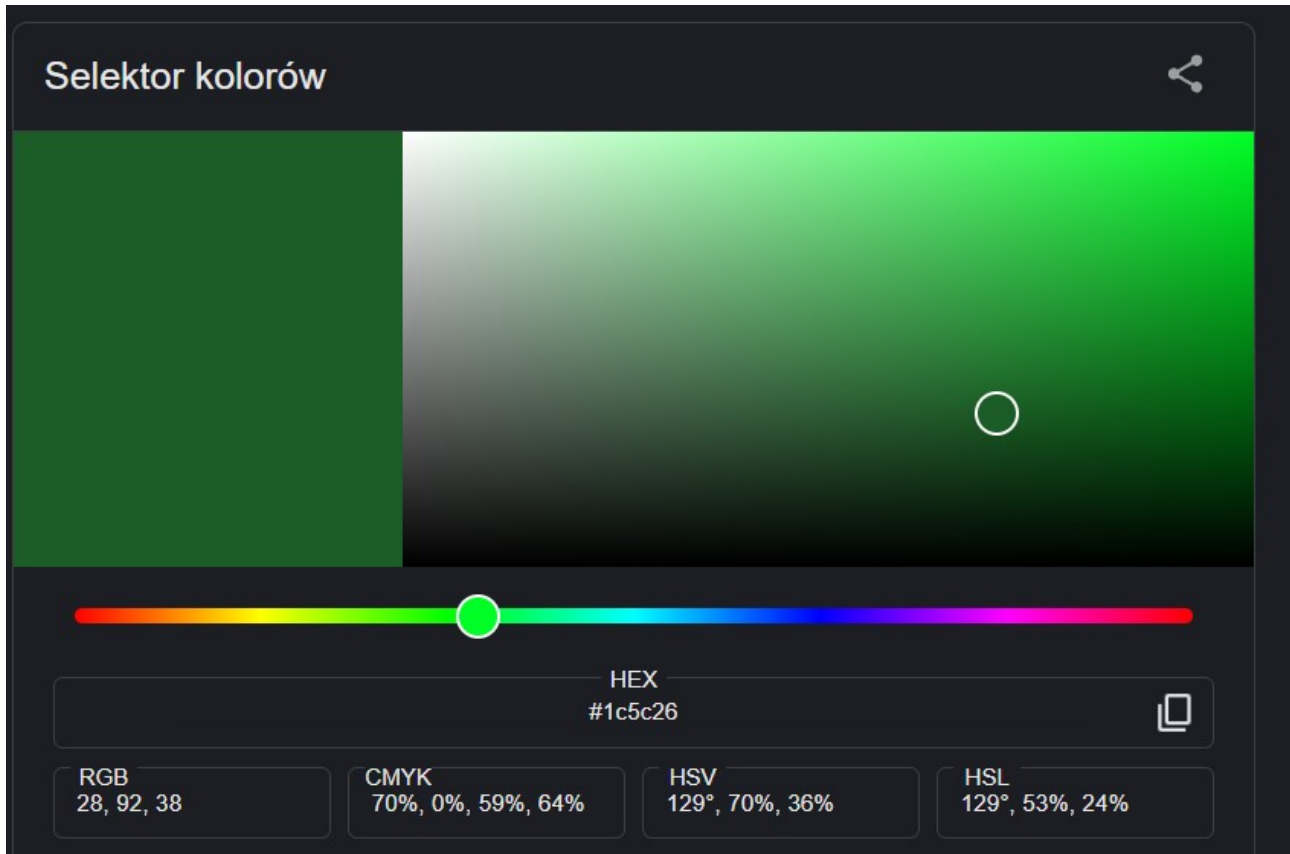
W wyniku przeprowadzonych testów kalibracyjnych, dla każdego z analizowanych kolorów cukierków wyznaczono charakterystyczne przedziały wartości składowych RGB oraz natężenia światła białego - Clear. Dane te stały się podstawą logiczną do zaprogramowania algorytmu decyzyjnego systemu. Szczególne znaczenie ma pomiaru stanu przy braku obiektu pod czujnikiem. Wykrycie przez sensor bardzo niskich wartości składowych pozwala systemowi na jednoznaczną identyfikację pustej komory pomiarowej. Funkcja ta jest kluczowa dla przepływu algorytmu: jeśli po wykonaniu sekwencji podawania czujnik nadal raportuje brak obiektu, program interpretuje to jako sytuację, w której cukierek nie został pobrany z lejka. W takim przypadku algorytm zostaje wstrzymany i następuje powrót do kroku pierwszego, czyli ponownej próby podania produktu. Rozwiązanie to zapobiega wykonywaniu „jałowych” cykli pracy przez serwomechanizm sortujący oraz eliminuje ryzyko błędnego klasyfikowania tła jako obiektu o konkretnej barwie. Przykładowe wyniki pomiarów, przedstawiające pierwsze dziesięć odczytów dla jednego z kolorów, zestawiono w tabeli 4.3.1.

Tabela 4.3.1. Przykładowe pomiary seryjne dla barwy zielonej odczytane za pomocą czujnika.

Nr pomiaru	R (Czerwony)	G (Zielony)	B (Niebieski)	C (Clear)
1	28	92	38	158
2	28	92	38	158
3	27	92	38	153
4	28	92	38	158
5	27	91	40	158
6	28	92	40	158
7	28	92	38	158
8	28	92	38	158
9	28	92	38	158
10	28	92	38	157

W celu ostatecznego potwierdzenia skuteczności przyjętej metody kalibracji, przeprowadzono próbę weryfikacyjną z wykorzystaniem ogólnodostępnego narzędzia diagnostycznego - cyfrowego selektora kolorów zintegrowanego z wyszukiwarką Google - *Google Color Picker*. Do symulatora wprowadzono uśrednione wartości uzyskane podczas pomiarów seryjnych, co pozwoliło na wizualizację barwy odpowiadającej odczytom R=28, G=92, B=38. Krótki test porównawczy potwierdził, że zdefiniowany przez czujnik profil cyfrowy odpowiada

faktycznej barwie badanego obiektu. Na podstawie wszystkich zebranych próbek wyliczono wartości średnie dla każdej kategorii. Zostały one zestawione w tabeli 4.3.2. i stanowią one bezpośredni punkt odniesienia dla instrukcji warunkowych if-else w algorytmie głównym.



Rys. 4.3.1. Weryfikacja odczytów kolorometrycznych w selektorze barw Google dla próbek koloru zielonego (RGB: 28, 58, 38) [Źródło: opracowanie własne na podstawie narzędzia Google Color Picker].

Tabela 4.3.2. Zestawienie średnich wartości składowych dla analizowanych barw i stanów układu.

Kolor obiektu	Średnie R	Średnie G	Średnie B	Średnie C
Czerwony	48	17	10	58
Żółty	115	108	35	170
Pomarańczowy	105	58	38	132
Zielony	28	92	38	134
Fioletowy	28	34	40	59
Niebieski	22	41	98	121
Brak obiektu	12	14	14	35

Na podstawie analizy danych pomiarowych zestawionych w tabeli wyznaczono precyzyjne progi decyzyjne, które pozwoliły na opracowanie optymalnego algorytmu sterującego.

- Brak obiektu (Pusto): Jeśli wartość składowej jasności wynosi $C \leq 40$, system rozpoznaje pustą komorę pomiarową, przerywa bieżący cykl i wymusza ponowną próbę pobrania cukierka z podajnika.
- Kolor fioletowy: W przypadku, gdy ogólna jasność mieści się w przedziale $40 < C < 75$, obiekt jest klasyfikowany jako fioletowy ze względu na charakterystyczną dla tej barwy wysoką absorpcję światła.
- Kolor żółty: Identyfikacja następuje, gdy jednocześnie składowa czerwona $R > 80$ oraz zielona $G > 80$, co pozwala na sprawne oddzielenie tej bardzo jasnej barwy od pozostałych próbek.
- Kolor zielony: Wykrycie następuje w momencie, gdy kanał zielony dominuje nad pozostałymi, czyli spełniony jest warunek relacyjny $G > R$ oraz $G > B$.
- Kolor niebieski: System przypisuje tę barwę, gdy składowa niebieska osiąga najwyższe wartości względem pozostałych kanałów ($B > G$ oraz $B > R$)
- Kolor pomarańczowy: Rozpoznawany jest przy dominacji czerwieni ($R > G$ oraz $R > B$), jeśli dodatkowo kanał zielony przekracza poziom $G > 33$.
- Kolor czerwony: Klasyfikacja następuje przy dominacji kanału czerwonego ($R > G$ oraz $R > B$), jeżeli jednocześnie zawartość zieleni pozostaje na niskim poziomie $G \leq 33$.

4.4. Opracowanie i analiza algorytmu sterującego w środowisku Arduino IDE

Głównym zadaniem oprogramowania stworzonego w środowisku Arduino IDE jest pełna kontrola nad procesem sortowania. Algorytm musi w odpowiednim czasie poruszać silnikami, odczytywać dane z czujnika kolorów i wyświetlać wyniki na ekranie. Całość została zaprojektowana tak, aby maszyna działała w sposób powtarzalny, a każdy ruch mechaniczny był zsynchronizowany z pomiarem optycznym. Program nie tylko wykonuje kolejne kroki, ale też na bieżąco sprawdza, czy proces przebiega poprawnie, np. czy w podajniku nie zabrakło materiału. Kod został podzielony na logiczne bloki, co ułatwiło jego testowanie i późniejszą kalibrację. W tej sekcji opisano poszczególne fragmenty oprogramowania, wyjaśniając, w jaki sposób surowe liczby z czujników są zamieniane na konkretne decyzje o kierunku sortowania cukierków.

Listing 4.4.1 Definicja stanów pracy oraz funkcja kontroli prędkości

```
1 #include <Wire.h>
2 #include <Servo.h>
3 #include <LiquidCrystal_I2C.h>
4 #include "Adafruit_TCS34725.h"
5
6
7 LiquidCrystal_I2C lcd(0x27, 16, 2);
8 Adafruit_TCS34725 tcs = Adafruit_TCS34725(TCS34725_INTEGRATIONTIME_50MS,
    TCS34725_GAIN_4X);
9
10 Servo servoPodajnik;
11 Servo servoSortujace;
12
13
14 const int POZ_POBOR = 90;
15 const int POZ_POMIAR = 35;
16 const int POZ_WYRZUT = 0;
17
18
19 int nCzer = 0, nPom = 0, nZol = 0, nZie = 0, nNie = 0, nFio = 0, nInny = 0;
20 int obecnaPozycja = POZ_POBOR;
21 int licznikPustych = 0;
```

Programowanie układu sterowania rozpoczęto od implementacji niezbędnych bibliotek zewnętrznych, które stanowią fundament komunikacji. Wykorzystanie biblioteki `Wire.h` jest kluczowe dla obsługi magistrali I2C, która umożliwia szeregową transmisję danych między Arduino a czujnikiem barw oraz wyświetlaczem LCD przy użyciu dwóch linii sygnałowych. Do precyzyjnego sterowania elementami mechanicznymi wykorzystano bibliotekę `Servo.h`, która zarządza generowaniem sygnałów o zmiennej szerokości impulsu, niezbędnych do ustalenia konkretnego kąta wychylenia serwomechanizmów. Warstwa wizualizacji opiera się na bibliotece `LiquidCrystal_I2C.h`, która współpracuje z konwerterem PCF8574, znacząco redukując liczbę wymaganych połączeń fizycznych. Za zaawansowaną analizę widma świetlnego odpowiada biblioteka `Adafruit_TCS34725.h`, umożliwiającą konfigurację wzmocnienia sygnału oraz czasu integracji pomiaru, co bezpośrednio przekłada się na czułość sensora w różnych warunkach oświetleniowych.

W strukturze kodu istotną rolę odgrywa definicja stałych parametrów geometrycznych urządzenia. Pozycja startowa wynosząca 90° odpowiada idealnemu zorientowaniu otworu pobierającego pod lejem zasypowym. Przesunięcie do wartości 35° lokuje cukierek w osi optycznej czujnika, natomiast osiągnięcie kąta 0° skutkuje grawitacyjnym opuszczeniem komory pomiarowej i zrzutem obiektu do separatora. Oprócz definicji ruchu, w sekcji tej zainicjowano zestaw zmiennych licznikowych przechowywanych w pamięci operacyjnej RAM mikrokontrolera. Zmienne te pełnią funkcję lokalnego rejestru zdarzeń, przechowując aktualną liczbę posortowanych jednostek w podziale na siedem zdefiniowanych kategorii barwnych. Tak zaprojektowana architektura zmiennych pozwala na dynamiczne raportowanie stanu pracy urządzenia na wyświetlaczu LCD.

Listing 4.4.2. Konfiguracja startowa i autotest systemu

```
49 void setup() {
50   lcd.init();
51   lcd.backlight();
52   lcd.print("SORTER GOTOWY");
53
54   servoSortujace.attach(2);
55   servoPodajnik.attach(3);
56
57   if (!tcs.begin()) {
58     lcd.clear();
59     lcd.print("BLAD SENSORA!");
60     while (1);
61   }
62
63   servoPodajnik.write(POZ_POBOR);
64   obecnaPozycja = POZ_POBOR;
65   servoSortujace.write(0);
66   delay(1000);
67 }
```

Funkcja `setup()` pełni rolę nadrzędnego kontrolera startu. W tej sekcji następuje przypisanie serwomechanizmów do konkretnych wyprowadzeń cyfrowych (piny 2 i 3) oraz nawiązanie komunikacji z czujnikiem barw. Kluczowym elementem jest mechanizm autodiagnostyki - jeśli czujnik nie zostanie wykryty na magistrali I2C, system wyświetla komunikat o błędzie i wstrzymuje dalszą pracę, co zapobiega awariom podczas próby segregacji.

Listing 4.4.3. Algorytm płynnego sterowania serwomechanizmem

```
24 void plynnyRuch(int cel, int czas) {  
25   if (obecnaPozycja < cel) {  
26     for (int i = obecnaPozycja; i <= cel; i++) {  
27       servoPodajnik.write(i);  
28       delay(czas);  
29     }  
30   } else {  
31     for (int i = obecnaPozycja; i >= cel; i--) {  
32       servoPodajnik.write(i);  
33       delay(czas);  
34     }  
35   }  
36   obecnaPozycja = cel;  
37 }
```

Zaimplementowana funkcja `plynnyRuch` stanowi kluczowy element sterowania ruchem, którego zadaniem jest programowe ograniczenie przyspieszenia serwomechanizmu. Funkcja przyjmuje dwa parametry: `cel`, czyli kąt docelowy oraz `czas` - opóźnienie w milisekundach między krokami. Logika rozpoczyna się od instrukcji warunkowej `if`, która porównuje zmienną `obecnaPozycja` z wartością zadaną, co pozwala określić kierunek obrotu wału silnika. W zależności od wyniku, uruchamiana jest pętla `for` zwiększająca lub zmniejszająca kąt o dokładnie jeden stopień w każdej iteracji. Wewnątrz pętli każdorazowo wywoływana jest metoda `servoPodajnik.write(i)`, która fizycznie zmienia położenie serwa o minimalny możliwy krok, a następująca po niej funkcja `delay(czas)` wstrzymuje wykonanie programu na określony interwał. To właśnie to opóźnienie definiuje prędkość obrotową - im wyższa wartość parametru `czas`, tym ruch jest wolniejszy i bardziej stabilny. Na samym końcu, po wyjściu z pętli, nowa wartość kąta zostaje przypisana do zmiennej globalnej `obecnaPozycja`, co pozwala systemowi na zachowanie ciągłości wiedzy o stanie mechanicznym tarczy przy kolejnych wywołaniach. Dzięki takiej strukturze, funkcja skutecznie eliminuje udary mechaniczne, co przekłada się na wysoką precyzję pozycjonowania cukierka w osi czujnika.

Listing 4.4.4. Moduł statystycznej obróbki sygnału z sensora

```
83 delay(3000);
84 float sumR = 0, sumG = 0, sumB = 0, sumC = 0;
85
86 for(int i = 0; i < 3; i++) {
87     uint16_t r_val, g_val, b_val, c_val;
88     tcs.getRawData(&r_val, &g_val, &b_val, &c_val);
89     sumR += r_val; sumG += g_val; sumB += b_val; sumC += c_val;
90     delay(50);
91 }
92
93 uint16_t r = sumR / 3; g = sumG / 3; b = sumB / 3; c = sumC / 3;
```

Proces odczytu danych rozpoczyna się od trzysekundowej pauzy, która pozwala na całkowite wygaszenie drgań tarczy podajnika. Stabilizacja ta jest niezbędna, aby sensor dokonał pomiaru nieruchomego obiektu, co eliminuje błędy wynikające z poruszenia cukierka. Następnie program wykonuje serię trzech pomiarów w odstępach 50 ms, korzystając ze zmiennych typu `uint16_t` jako tymczasowych schowków na surowe informacje z czujnika. Wartości te są sumowane do zmiennych typu `float`, co chroni przed błędami obliczeniowymi przy bardzo jasnych próbkach. Ostatecznie system wylicza średnią arytmetyczną dla każdego kanału, dzieląc sumy przez liczbę wykonanych prób. Dzięki temu algorytm pracuje na „wygładzonych” wynikach, pozbawionych przypadkowych zakłóceń czy chwilowych odbłasków światła. Tak przygotowane wartości stanowią stabilną podstawę dla instrukcji warunkowych, co w praktyce zapewnia wysoką powtarzalność odczytów i znacznie zmniejsza ryzyko, że urządzenie pomyli kolory.

Listing 4.4.5. Implementacja algorytmu wyboru koloru i kąta separatora

```
105
106 int katKubelka = 0;
107
108
109 if (c < 75) {
110     katKubelka = 90; nFio++; // Fioletowy
111 }
112 else if (r > 80 && g > 80) {
113     katKubelka = 45; nZol++; // Żółty
114 }
115 else if (g > r && g > b) {
116     katKubelka = 60; nZie++; // Zielony
117 }
118 else if (b > g && b > r) {
119     katKubelka = 75; nNie++; // Niebieski
120 }
121 else if (r > g && r > b) {
122     |
123     if (g > 33) { katKubelka = 30; nPom++; } // Pomarańczowy
124     else { katKubelka = 15; nCzer++; } // Czerwony
125 }
126 else {
127     katKubelka = 105; nInny++; // Kategoria dla barw nierozpoznanych
128 }
```

Po uzyskaniu uśrednionych wartości składowych światła, program przechodzi do fazy interpretacji wyników, która determinuje dalsze działanie mechanizmu sortującego. Klasyfikacja barw nie opiera się na sztywnych zakresach liczbowych, lecz na analizie dominacji konkretnych kanałów RGB względem siebie. Przykładowo, kolor zielony jest identyfikowany w momencie, gdy składowa zielona osiąga najwyższą wartość w porównaniu do pozostałych dwóch kanałów. Najbardziej zaawansowanym elementem tej sekcji jest różnicowanie barw o zbliżonym widmie, takich jak czerwona i pomarańczowa. Zastosowano tutaj połączoną instrukcję warunkową, która po stwierdzeniu przewagi kanału czerwonego, dodatkowo weryfikuje poziom kanału zielonego; jego podwyższona wartość klasyfikuje obiekt jako pomarańczowy, co znacząco podnosi precyzję segregacji. Każda pozytywna weryfikacja koloru kończy się przypisaniem odpowiedniego kąta do

zmiennej sterującej dolnym serwomechanizmem oraz aktualizacją odpowiedniego licznika w pamięci RAM.

Listing 4.4.6. Implementacja obsługi zatorów i blokady systemowej

```
94
95  if (c <= 40) {
96      licznikPustych++;
97
98
99      if (licznikPustych >= 5) {
100          lcd.clear();
101          lcd.print("ZATOR SYSTEMU");
102          while(1);
103      }
104      return;
105  }
106
107
108  licznikPustych = 0;
```

Integralną częścią algorytmu sterującego jest moduł nadzoru nad ciągłością podawania materiału. Funkcja ta ma na celu odróżnienie standardowej pracy urządzenia od sytuacji awaryjnych, takich jak całkowity brak cukierków w zasobniku lub zablokowanie się obiektu w leju grawitacyjnym. System monitoruje kanał jasności całkowitej i w przypadku, gdy odczytana wartość jest niższa lub równa 40 jednostkom, interpretuje to jako wykrycie pustej komory pomiarowej. Aby uniknąć fałszywych alarmów, zaimplementowano licznik prób, który pozwala maszynie na wykonanie dodatkowych cykli w celu samoczynnego udrożnienia blokady. Dopiero po pięciu kolejnych nieudanych próbach detekcji obiektu, mikrokontroler przechodzi w tryb awaryjny, generując komunikat na wyświetlaczu LCD i wstrzymując pracę silników w celu ochrony podzespołów przed jałową eksploatacją. Zastosowanie zmiennej `licznikPustych` pozwala na zachowanie autonomii urządzenia w przypadku drobnych nieregularności w podawaniu materiału. Dopiero trwała utrata sygnału z sensora optycznego aktywuje pętlę blokującą `while(1)`, która wymaga manualnej ingerencji operatora. Rozwiązanie to stanowi kluczowy element

bezpieczeństwa, zabezpieczając serwomechanizmy oraz strukturę mechaniczną sortera przed niepotrzebnym zużyciem.

Listing 4.4.7. Funkcja aktualizacji interfejsu HMI na wyświetlaczu LCD

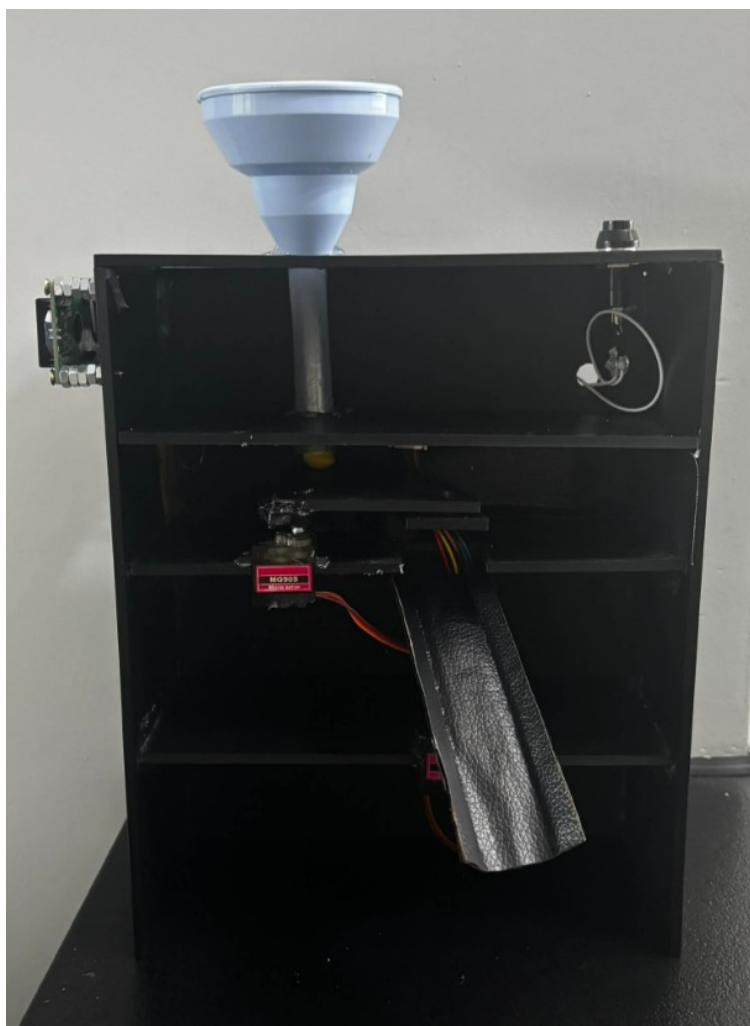
```
40 void wyswietlStatystyki() {  
41     lcd.clear(); // Czyszczenie bufora wyświetlacza  
42  
43     // Ustawienie etykiet i wartości dla pierwszego wiersza  
44     lcd.setCursor(0, 0);  
45     lcd.print("R"); lcd.print(nCzer);  
46     lcd.print(" O"); lcd.print(nPom);  
47     lcd.print(" Y"); lcd.print(nZol);  
48     lcd.print(" G"); lcd.print(nZie);  
49  
50     // Ustawienie etykiet i wartości dla drugiego wiersza  
51     lcd.setCursor(0, 1);  
52     lcd.print("B"); lcd.print(nNie);  
53     lcd.print(" V"); lcd.print(nFio);  
54     lcd.print(" ?"); lcd.print(nInny);  
55 }
```

Ostatnim modulem algorytmu sterującego jest funkcja odpowiedzialna za prezentację danych procesowych na wyświetlaczu 16x2. Zadaniem tej sekcji jest przekształcenie surowych danych liczbowych przechowywanych w pamięci RAM mikrokontrolera na czytelną dla operatora formę tekstową. Ze względu na ograniczoną przestrzeń roboczą matrycy LCD, zaprojektowano układ komunikatów, który pozwala na jednoczesne monitorowanie wszystkich siedmiu kategorii kolorystycznych. Funkcja wykorzystuje precyzyjne adresowanie kursora, co zapewnia stałe położenie etykiet i zapobiega migotaniu ekranu podczas częstych aktualizacji danych. Stosowanie skrótów literowych - R dla czerwonego, O dla pomarańczowego itd., pozwoliło na zachowanie wysokiej czytelności interfejsu przy zachowaniu minimalizmu formy. Każdorazowe wywołanie tej funkcji następuje po zakończeniu pełnego cyklu sortowania, co gwarantuje natychmiastowy wgląd w wydajność. Tak zaprojektowana warstwa prezentacji danych stanowi zamknięcie pętli sterowania.

4.5. Finalizacja i wygląd sortera

Po zintegrowaniu wszystkich modułów elektronicznych z konstrukcją mechaniczną oraz wgraniu oprogramowania sterującego, urządzenie osiągnęło pełną gotowość operacyjną. Sorter charakteryzuje się kompaktową budową, w której kluczowe elementy, czyli sensor koloru oraz tarcza podajnika, są odizolowane od zmiennego oświetlenia zewnętrznego, co dodatkowo stabilizuje proces odczytu.

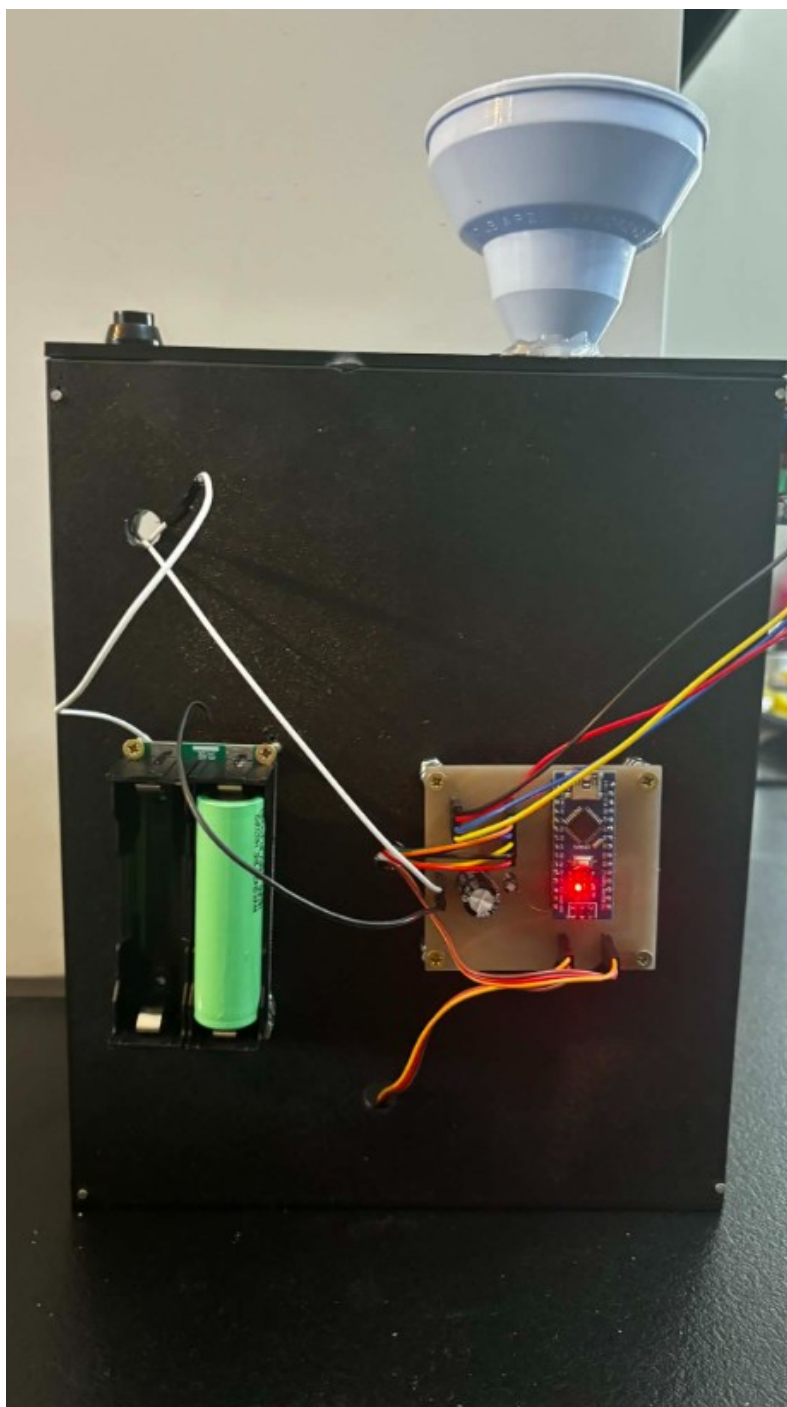
Poniższa fotografia (Rys. 4.5.1) przedstawia kompletny prototyp w trakcie aktywnego cyklu pracy. Na zdjęciu widoczny jest moment poboru cukierka oraz reakcja separatora dolnego, który ustawia się pod odpowiednim kątem, aby skierować obiekt do właściwego pojemnika. Estetyka wykonania za pomocą druku 3D pozwoliła na zachowanie sztywności konstrukcji przy jednoczesnej możliwości łatwego serwisowania podzespołów.



Rys. 4.5.1. Kompletny sorter do cukierków Skittles - widok od przodu.



Rys. 4.5.2. Kompletny sorter do cukierków Skittles - widok modułu wyświetlającego.



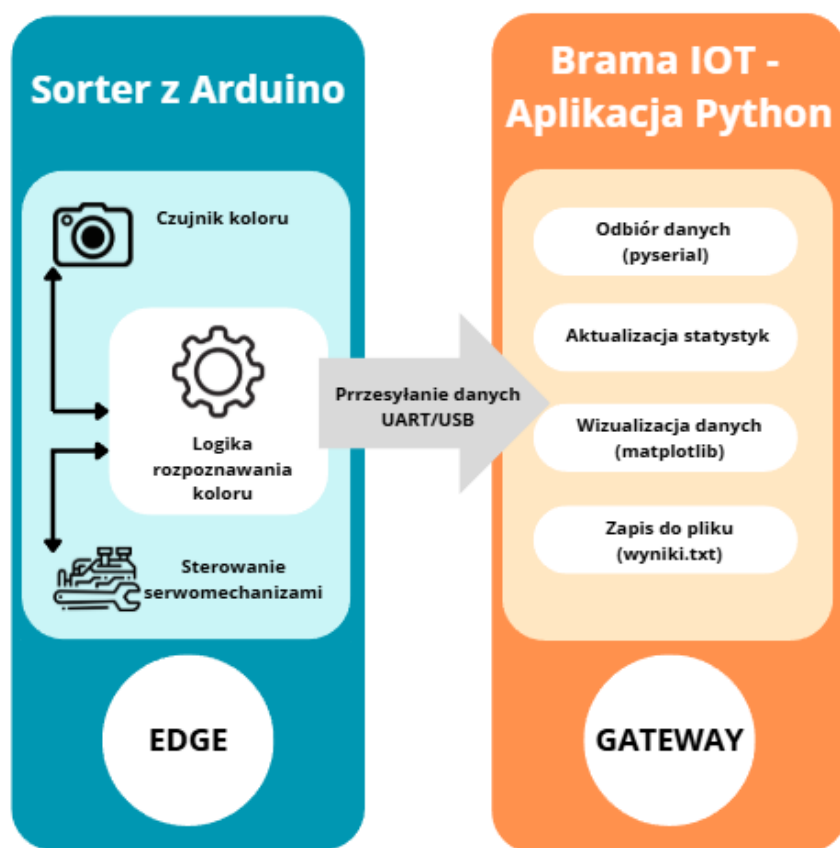
Rys. 4.5.3. Kompletny sorter do cukierków Skittles - widok z tyłu, eksponujący zasilanie oraz sposób podłączenia urządzenia do płytki PCB.

5. INTEGRACJA UKŁADU STERUJĄCEGO Z MODUŁEM PREZENTACJI WYNIKÓW DZIAŁANIA URZĄDZENIA

Współczesna automatyka przemysłowa opiera się na ścisłej integracji maszyn z zaawansowanymi systemami analizy danych. W niniejszym projekcie proces ten zrealizowano poprzez połączenie mikrokontrolera Arduino z aplikacją napisaną w języku Python, co pozwoliło przekształcić fizyczny sorter w inteligentny system pracujący w architekturze Edge Computing [15]. Kluczem do efektywności takiego rozwiązania jest następujący podział zadań: Arduino pełni funkcję warstwy wykonawczej, realizując operacje, takie jak odczyt barw, ich klasyfikacja oraz bezpośrednie sterowanie serwomechanizmami. Zapewnia to natychmiastową reakcję mechaniki bez opóźnień, które mogłyby wystąpić przy sterowaniu bezpośrednio z poziomu systemu operacyjnego komputera. Z kolei aplikacja w języku Python pełni rolę bramy IoT oraz warstwy analitycznej, która odbiera gotowe wyniki przez port szeregowy. Odpowiada ona za dynamiczną wizualizację statystyk w formie graficznej oraz trwałą archiwizację danych w plikach tekstowych. Dzięki takiemu podejściu użytkownik zyskuje nie tylko zdalny nadzór nad wydajnością urządzenia, ale także dostęp do precyzyjnych logów procesowych, co stanowi wstęp do nowoczesnego zarządzania produkcją.

5.1. Architektura obliczeń krawędziowych (Edge Computing) w systemie IoT

Projekt oparto na modelu Edge Computing, czyli obliczeniach wykonywanych bezpośrednio na krawędzi systemu. W praktyce oznacza to, że najważniejsze decyzje, takie jak rozpoznanie koloru cukierka i sterowanie silnikami zapadają od razu na Arduino, a nie na komputerze. Dzięki temu, że logika sterowania znajduje się na samym urządzeniu, system pracuje bardzo szybko i bez opóźnień, co pozwala tarczy poruszać się płynnie i precyzyjnie trafiać do odpowiednich pojemników. W takim układzie komputer z aplikacją Python pełni jedynie funkcję Bramy IoT, zwanej Gateway. Zamiast wysyłać co chwilę setki surowych liczb z czujnika, Arduino przesyła tylko krótką informację o wyniku, np. nazwę koloru po każdym zakończonym cyklu. Takie rozwiązanie sprawia, że połączenie między urządzeniami nie jest przeciążone, a cały system działa stabilnie i nie zawiesza się, nawet jeśli komputer w tym czasie wykonuje inne zadania.



Rys. 5.1.1. Schemat blokowy przepływu danych w architekturze Edge-Gateway. (Opracowanie własne na podstawie programu Canva).

5.2. Rozbudowa oprogramowania Arduino o moduł transmisji danych

Kluczowym etapem integracji systemu było zaimplementowanie modułu odpowiedzialnego za komunikację szeregową UART. Aby umożliwić komputerowi nadrzędnemu analizę pracy sortera, oprogramowanie mikrokontrolera rozbudowano o sekcję formatowania i wysyłki danych procesowych. Wykorzystano prędkość transmisji 9600 bodów, co zapewnia optymalną szybkość.

Dane są generowane w formacie CSV w postaci wartości oddzielonych przecinkiem, co stanowi uniwersalny standard wymiany informacji między różnymi platformami programistycznymi. W każdym cyklu pracy, po rozpoznaniu koloru i wykonaniu ruchu przez serwa, Arduino buduje pojedynczą „ramkę danych”. Zawiera ona aktualne stany wszystkich liczników jako podział na ich kolory. Ramka kończy się znakiem nowej linii (\n), który dla skryptu w języku Python jest sygnałem do zakończenia odczytu i rozpoczęcia aktualizacji statystyk.

Listing 5.2.1. Implementacja przesyłu danych procesowych w języku C++

```
1 Serial.print(nCzer); Serial.print(",");  
2 Serial.print(nPom); Serial.print(",");  
3 Serial.print(nZol); Serial.print(",");  
4 Serial.print(nZie); Serial.print(",");  
5 Serial.print(nNie); Serial.print(",");  
6 Serial.print(nFio); Serial.print(",");  
7 Serial.println(nInny);
```

Wykorzystanie formatu CSV pozwala na uniknięcie przesyłania skomplikowanych obiektów, co znacząco odciąża jednostkę centralną mikrokontrolera i pozwala mu skupić się na sterowaniu w czasie rzeczywistym. Jak wskazują wymienione powyżej źródła, takie podejście do architektury IoT pozwala na skuteczne monitorowanie urządzeń bez ryzyka utraty stabilności systemu wykonawczego.

5.3. Struktura i analiza modułowa skryptu Python

Aplikację nadrzędną podzielono na cztery główne segmenty, z których każdy realizuje inne zadanie w procesie monitorowania i archiwizacji danych.

Listing 5.3.1. Konfiguracja parametrów połączenia i struktur danych

```
1 import serial
2 import time
3 import matplotlib.pyplot as plt
4 from datetime import datetime
5
6
7 PORT_ARDUINO = 'COM3'
8 BAUDRATE = 9600
9 NAZWA_PLIKU = 'raporty_sesji.txt'
10
11 # Inicjalizacja liczników i wyglądu wykresu
12 mozliwe_kolory = ['CZERWONY', 'ZIELONY', 'NIEBIESKI', 'ZOLTY', 'POMARANCZOWY',
13                  'FIOLETOWY', 'NIEZNANY']
14 kolory_wykresu = ['red', 'green', 'blue', 'yellow', 'orange', 'purple', 'gray']
15 licznik_kolorow = {k: 0 for k in mozliwe_kolory}
16 czas_startu = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

Pierwszy segment skryptu stanowi fundament logiki analitycznej, odpowiadając za import niezbędnych bibliotek, takich jak `pyserial` do komunikacji oraz `matplotlib` do wizualizacji, a także za ścisłą konfigurację parametrów transmisyjnych. Kluczowym elementem tej sekcji jest zdefiniowanie stałych, takich jak port COM i prędkość 9600 bodów, które muszą być identyczne z ustawieniami mikrokontrolera, aby zapobiec błędom w odczycie danych. Równie istotne jest przygotowanie struktur danych w postaci listy dopuszczalnych kolorów oraz słownika zliczającego, co pełni rolę „filtra bezpieczeństwa”- dzięki temu program akceptuje i zlicza jedynie te komunikaty, które są spójne z logiką rozpoznawania barw zaprogramowaną w Arduino. Dodatkowo w tej części inicjowany jest znacznik czasu rozpoczęcia sesji, co pozwala na późniejsze określenie wydajności systemu w generowanym raporcie końcowym.

Listing 5.3.2. Pętla główna odpowiedzialna za odbiór i filtrowanie danych

```
1 try:
2     print(f"Łączenie z Arduino na porcie {PORT_ARDUINO}...")
3     ser = serial.Serial(PORT_ARDUINO, BAUDRATE, timeout=1)
4     time.sleep(2)
5     print(f"--- ROZPOCZĘTO SESJĘ: {czas_startu} ---")
6
7     while True:
8         if ser.in_waiting > 0:
9             # Odczyt i czyszczenie danych z portu szeregowego
10            linia = ser.readline().decode('utf-8').strip()
11
12            # Weryfikacja spójności odebranego koloru
13            if linia in licznik_kolorow:
14                licznik_kolorow[linia] += 1
15                print(f"Wykryto: {linia}")
```

Drugi segment oprogramowania odpowiada za utrzymanie ciągłej łączności oraz interpretację komunikatów przesyłanych przez mikrokontroler w czasie rzeczywistym. Po nawiązaniu fizycznego połączenia i odczekaniu dwóch sekund na stabilizację portu szeregowego, program wchodzi w nieskończoną pętlę `while True`, w której stale monitoruje stan bufora wejściowego za pomocą funkcji `ser.in_waiting`. W momencie pojawienia się danych, skrypt odczytuje całą linię tekstu, dokonuje jej dekodowania z formatu bajtowego na czytelny dla człowieka ciąg znaków oraz usuwa zbędne białe znaki za pomocą metody `strip()`. Kluczowym mechanizmem jest tutaj instrukcja warunkowa, która sprawdza, czy odebrany napis znajduje się na liście dopuszczalnych kolorów i dopiero po pozytywnej weryfikacji następuje inkrementacja odpowiedniego licznika w słowniku oraz wyświetlenie potwierdzenia w konsoli systemowej. Taka architektura zapewnia wysoką odporność na zakłócenia transmisji, gdyż wszelkie niekompletne lub błędne komunikaty są automatycznie ignorowane.

Listing 5.3.3. Implementacja dynamicznego odświeżania wykresu procesowego

```
1 # Aktualizacja Wykresu
2     ax.clear()
3     wartosci = [licznik_kolorow[k] for k in mozliwe_kolory]
4     bars = ax.bar(mozliwe_kolory, wartosci, color=kolory_wykresu)
5     ax.set_title(f"Sesja od: {czas_startu}")
6     ax.set_ylabel("Liczba sztuk")
7
8
9     for bar in bars:
10         height = bar.get_height()
11         if height > 0:
12             ax.text(bar.get_x() + bar.get_width() / 2., height,
13                     f'{int(height)}', ha='center', va='bottom')
14
15     plt.pause(0.1)
```

Trzeci segment kodu koncentruje się na przetwarzaniu zgromadzonych w pamięci danych na interaktywną formę graficzną, co pozwala na bieżące śledzenie wydajności sortera. Wykorzystując bibliotekę `matplotlib` w trybie interaktywnym program przy każdym nowym zdarzeniu czyści poprzedni widok za pomocą funkcji `ax.clear()`, a następnie generuje aktualny wykres słupkowy na podstawie bieżących stanów słownika liczników. Aby jeszcze poprawić czytelność wizualizacji, zastosowano pętlę iterującą po nowo utworzonych słupkach, która oblicza ich wysokość i nanosi etykiety liczbowe bezpośrednio nad nimi, eliminując konieczność analizy osi rzędnych. Cały proces odświeżania jest zoptymalizowany przez funkcję `plt.pause(0.1)`, która zapewnia płynność animacji wykresu oraz stabilność interfejsu użytkownika nawet przy częstym napływie danych z mikrokontrolera. Takie podejście umożliwia natychmiastową weryfikację poprawności działania urządzenia.

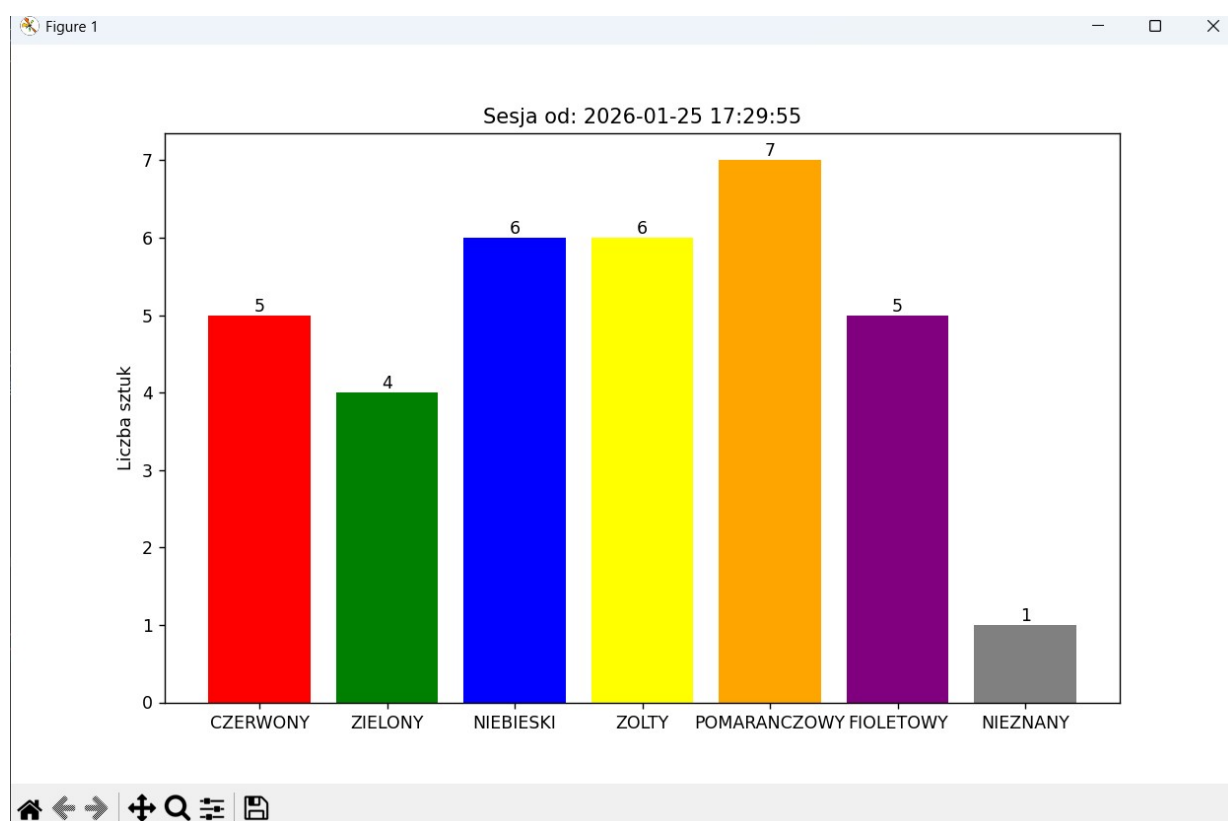
Listing 5.3.4. Procedura bezpiecznego zakończenia sesji i zapisu raportu

```
1 except KeyboardInterrupt:
2
3     print("\n\nKończenie sesji... Zapisywanie raportu...")
4
5     czas_konca = datetime.now().strftime("%H:%M:%S")
6     suma_cukierkow = sum(licznik_kolorow.values())
7
8
9     with open(NAZWA_PLIKU, "a", encoding="utf-8") as f:
10         f.write("\n" + "=" * 40 + "\n")
11         f.write(f" RAPORT SESJI SORTOWANIA\n")
12         f.write(f" Start: {czas_startu} | Koniec: {czas_konca}\n")
13         f.write("=" * 40 + "\n")
14
15         for kolor, ilosc in licznik_kolorow.items():
16             if ilosc > 0:
17                 f.write(f" - {kolor.ljust(15)} : {ilosc} szt.\n")
18
19         f.write("-" * 40 + "\n")
20         f.write(f" ŁĄCZNIE POSORTOWANO: {suma_cukierkow} szt.\n")
21         f.write("=" * 40 + "\n\n")
22
23     ser.close()
24     plt.ioff()
25     plt.show()
```

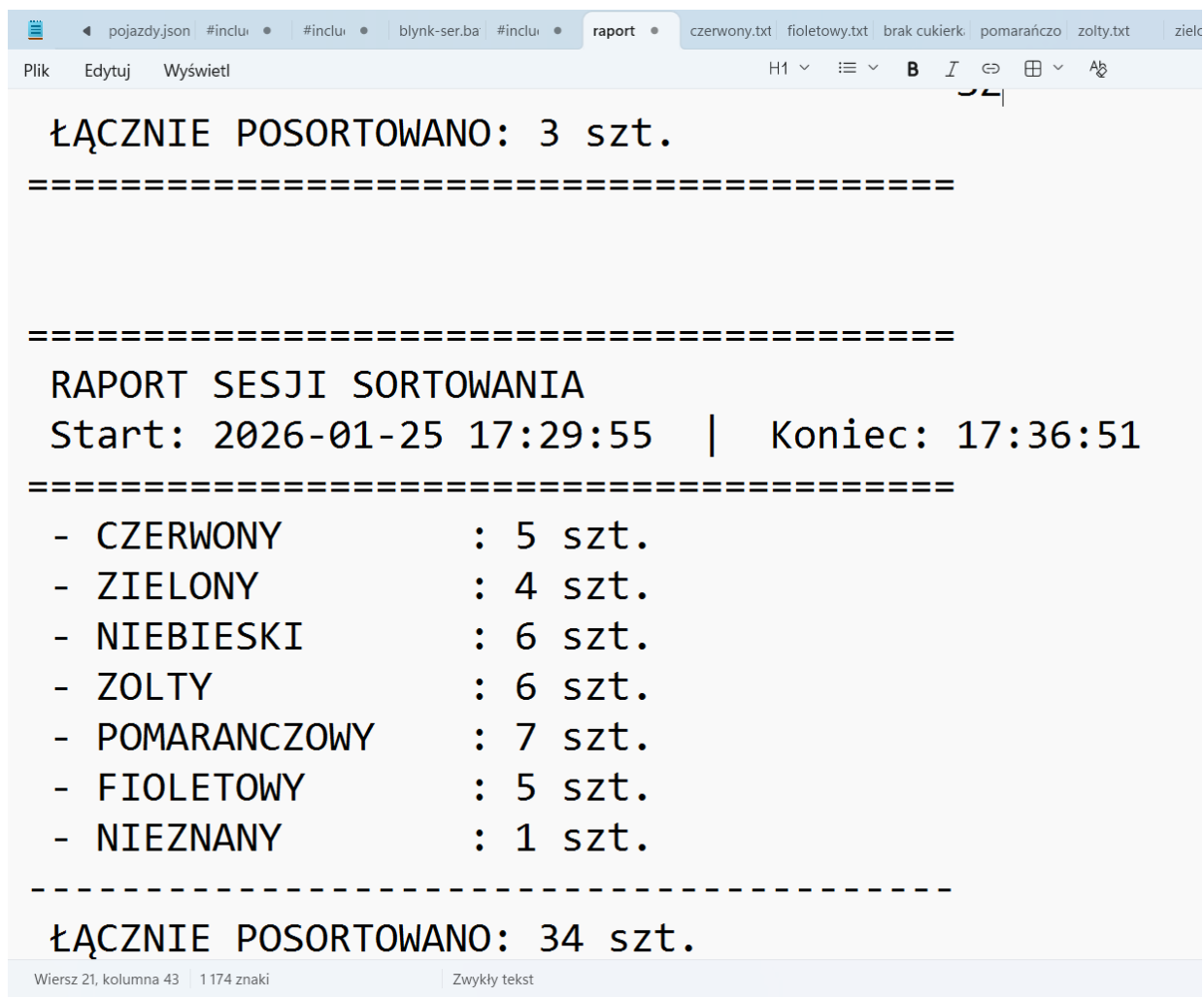
Ostatni segment aplikacji odpowiada za krytyczne operacje związane z bezpiecznym zamknięciem systemu oraz zapisywaniem zgromadzonych danych procesowych. Wykorzystanie bloku `except KeyboardInterrupt` pozwala programowi wykryć moment, w którym użytkownik ręcznie przerywa pracę skryptu, co inicjuje procedurę generowania raportu końcowego zamiast gwałtownego zamknięcia aplikacji. W ramach tej procedury skrypt oblicza czas zakończenia sesji oraz sumuje wszystkie zliczone obiekty, a następnie otwiera plik *raporty_sesji.txt* w trybie dopisywania, co zapobiega nadpisywaniu wyników z poprzednich dni. Dzięki zastosowaniu metod formatowania tekstu, takich jak `ljust()`, raport w pliku tekstowym przyjmuje formę czytelnej tabeli, w której uwzględniane są jedynie te kolory, których licznik był większy od zera/ Całość procesu kończy się bezpiecznym zwolnieniem zasobów portu szeregowego oraz ostatecznym wyświetleniem wykresu w trybie statycznym, co stanowi kompletne zamknięcie cyklu monitorowania.

5.4. Dokumentacja wizualna modułu prezentacji

Ostatnim etapem wdrożenia oprogramowania była weryfikacja poprawności przepływu informacji między urządzeniem brzegowym a aplikacją nadrzędną. Jak pokazują załączone zrzuty ekranu, system w pełni realizuje założone funkcje monitorowania i dokumentowania procesu sortowania w czasie rzeczywistym.



Rys. 5.4.1. Interfejs graficzny aplikacji monitorującej z wynikami końcowymi sesji.



```
ŁĄCZNIE POSORTOWANO: 3 szt.
=====

=====
RAPORT SESJI SORTOWANIA
Start: 2026-01-25 17:29:55 | Koniec: 17:36:51
=====
- CZERWONY           : 5 szt.
- ZIELONY            : 4 szt.
- NIEBIESKI          : 6 szt.
- ZOLTY              : 6 szt.
- POMARANCZOWY       : 7 szt.
- FIOLETOWY          : 5 szt.
- NIEZNANY           : 1 szt.
-----
ŁĄCZNIE POSORTOWANO: 34 szt.
```

Rys. 5.4.1. Przykładowy raport sesji wygenerowany w pliku tekstowym po zatrzymaniu programu.

Aplikacja Python, działająca jako brama IoT, skutecznie interpretuje przesyłane komunikaty tekstowe. Na rysunku 5.4.1. widoczny jest finałowy stan wykresu po zakończeniu sesji, w której posortowano 34 obiekty. Poprawne przypisanie słupków do kolorów - np. 7 sztuk pomarańczowych, 6 niebieskich, świadczy o wysokiej stabilności połączenia szeregowego oraz braku opóźnień w odświeżaniu interfejsu graficznego. Każda zmiana stanu licznika na Arduino jest natychmiastowo pokazywana na ekranie komputera, co pozwala operatorowi na bieżącą kontrolę pracy maszyny.

Równolegle z wizualizacją, system dba o trwałość zgromadzonych informacji. Na rysunku 5.4.2. przedstawiono strukturę automatycznie generowanego raportu w formacie tekstowym. Program nie tylko zlicza cukierki, ale również precyzyjnie oznacza ramy czasowe sesji, co widać w sekcjach „Start” i „Koniec”. Zastosowanie prostego pliku tekstowego z czytelnym formatowaniem sprawia, że dane są gotowe do odczytu bez konieczności posiadania specjalistycznego oprogramowania.

Podsumowując działanie warstwy programowej, połączenie szybkiej logiki sterowania na Arduino z analitycznymi możliwościami Pythona pozwoliło na stworzenie stabilnego inteligentnego systemu – maszyna potrafi nie tylko fizycznie rozdzielać obiekty jako urządzenie lokalne, ale również potrafi dostarczać kompletnych danych o przebiegu tego procesu.

6. TESTOWANIE I OCENA DZIAŁANIA WYKONANEGO URZĄDZENIA ORAZ ANALIZA WYNIKÓW PRACY SYSTEMU

W celu potwierdzenia poprawnego działania zbudowanego prototypu oraz weryfikacji poprawności zaimplementowanych algorytmów, przeprowadzono kompleksowe testy sprawdzające. Proces walidacji miał na celu ocenę stabilności współpracy mikrokontrolera z układami wykonawczymi oraz precyzji przesyłu danych do aplikacji nadrzędnej. Wszystkie testy realizowano w seriach po 50 obiektów testowych dla każdej kategorii, lub po 20 sztuk danego koloru, co pozwoliło na uzyskanie reprezentatywnych wyników statystycznych i wykluczenie przypadkowych błędów mechanicznych.

W ramach rozdziału poddano analizie następujące obszary:

- Testy funkcjonalne - weryfikacja poprawności ruchów mechanicznych i komunikacji z użytkownikiem.
- Testy kalibracyjne czujnika - optymalizacja progów chromatyczności na podstawie zebranych próbek danych.
- Testy skuteczności klasyfikacji - ocena celności sortowania dla różnych grup kolorystycznych.
- Testy archiwizacji danych - sprawdzenie poprawności generowanych raportów sesji w formacie tekstowym.

6.1. Testy funkcjonalne systemu

Testy funkcjonalne stanowiły pierwszy etap weryfikacji urządzenia i miały na celu sprawdzenie, czy każdy komponent elektroniczny oraz mechaniczny reaguje zgodnie z programem sterującym w powtarzalnych cyklach pracy. Badania przeprowadzono w seriach po 50 cykli dla każdej operacji, monitorując płynność ruchów oraz stabilność odczytów na wyświetlaczu LCD. W tabeli 6.1.1 zestawiono wyniki obserwacji dla kluczowych modułów systemu:

Tabela 6.1.1. Wyniki testów funkcjonalnych podzespołów (seria 50 powtórzeń).

Element systemu	Zakres testu	Skuteczność	Uwagi
Serwo Podajnika	Cykl: pobór - pomiar - wyrzut	100%	Funkcja płynnyRuch zapewniła brak gubienia pozycji.
Serwo Sortujące	Ustawienie rynn pod zadany kąt	100%	Stabilne utrzymanie kąta (20°–160°) podczas zrzutu obiektu.
Czujnik Koloru	Akwizycja danych przy 50ms integracji	100%	Brak błędów komunikacji I2C podczas 50-elementowych serii.
Wyświetlacz LCD	Odświeżanie statystyk i komunikatów	100%	Dane prezentowane bez opóźnień względem cyklu pracy.
Transmisja UART	Przesył danych do bramy IoT (Python)	100%	Wszystkie 50 ramek na serię odebrano poprawnie przez port COM.

Podczas testów funkcjonalnych szczególną uwagę zwrócono na stabilność zasilania przy jednoczesnej pracy dwóch serwomechanizmów. System zachował pełną sprawność, co potwierdziło poprawność doboru komponentów oraz logiki sterowania umieszczonej na urządzeniu brzegowym.

6.2. Testy kalibracyjne i weryfikacja stabilności czujnika koloru

Kluczowym elementem warstwy pomiarowej systemu jest czujnik TCS34725, którego zadaniem jest precyzyjna identyfikacja barwy obiektu w warunkach rzeczywistych. Podczas testów szczególną uwagę poświęcono stabilności odczytów w zależności od natężenia oświetlenia zewnętrznego. Przeprowadzono serie pomiarowe po 50 obiektów w dwóch skrajnych scenariuszach: przy pełnym oświetleniu laboratoryjnym oraz przy świetle zgaszonym. Analiza zebranych danych w Tabeli 6.2.1. wykazała, że wartości składowych chromatycznych (proporcje pR, pG, pB) pozostawały zbliżone w obu przypadkach. Wynik ten potwierdza poprawność konstrukcji komory optycznej oraz osłony czujnika, która skutecznie eliminuje wpływ światła rozproszonego na proces decyzyjny mikrokontrolera.

Tabela 6.2.1. Porównanie stabilności odczytów dla wybranych próbek (średnie z serii 50 pomiarów).

Lp.	Obiekt testowy	Światło zewnętrzne	pR (%)	pG (%)	pB (%)	Stan systemu
1.	Czerwony	Włączone	46,8	27,7	25,5	Stabilny
2.	Czerwony	Wyłączone	43,7	24,8	29,2	Stabilny
3.	Pomarańczowy	Włączone	53,5	27,2	19,3	Stabilny
4.	Pomarańczowy	Wyłączone	50,4	25,3	14,3	Stabilny
5.	Żółty	Włączone	38,5	41,7	19,8	Stabilny
6.	Żółty	Wyłączone	40,6	43,6	20,8	Stabilny
7.	Zielony	Włączone	24,0	52,0	24,0	Stabilny
8.	Zielony	Wyłączone	23,1	51,9	22,0	Stabilny
9.	Niebieski	Włączone	10,6	39,4	50,0	Stabilny
10.	Niebieski	Wyłączone	9,6	29,5	42,9	Stabilny
11.	Fioletowy	Włączone	27,5	33,3	39,2	Stabilny
12.	Fioletowy	Wyłączone	23,6	29,2	34,2	Stabilny
13.	Brak obiektu	Włączone	25,8	38,7	35,5	Tłó
14.	Brak obiektu	Wyłączone	25,9	34,6	33,5	Tłó

Wyniki zawarte w tabeli jednoznacznie wskazują, że zmiana oświetlenia zewnętrznego nie wpływa znacząco na proces decyzyjny urządzenia. Różnice na poziomie części procenta są pomijalne i wynikają jedynie z naturalnego szumu czujnika, a nie z nieszczelności komory optycznej. Potwierdza to, że system jest przygotowany do pracy w zmiennych warunkach środowiskowych.

6.3. Ocena sprawności mechanicznej układu rozdzielającego

Kolejnym etapem badań była weryfikacja precyzji działania rynny sortującej, sterowanej przez drugi serwomechanizm. Test ten miał na celu sprawdzenie, czy fizyczne ustawienie rynny pod wyznaczonymi kątami (20° , 50° , 70° , 90° , 110° , 135° oraz 160°) gwarantuje powtarzalne trafianie obiektów do właściwych przegród. Badanie przeprowadzono w serii 50 prób, monitorując trajektorię lotu cukierka po zrzucie z tarczy podajnika.

Wyniki testu mechanicznego:

Wyniki przeprowadzonej serii 50 prób operacyjnych wykazały wysoką powtarzalność działania układu sortującego, osiągając sprawność mechaniczną na poziomie 94%. W toku analizy odnotowano trzy przypadki niezgodności, których szczegółowa diagnoza pozwoliła na sformułowanie następujących wniosków:

- Nadmierna dynamika procesu zrzutu: Główną przyczyną błędów nie były nieprawidłowe nastawy katowe, lecz zbyt gwałtowny ruch tarczy podajnika. Podczas przejścia z pozycji pomiarowej (35°) do pozycji wyrzutu (0°), obiekt nabierał zbyt dużej energii kinetycznej, co skutkowało jego uderzeniem o krawędź rynny i niekontrolowanym rykoszetowaniem poza obszar docelowego zasobnika.
- Problemy z synchronizacją czasową: Zaobserwowana bezwładność układu wskazuje, że standardowy czas opóźnienia pomiędzy ustawieniem rynny sortującej a momentem zrzutu był w skrajnych przypadkach niewystarczający, co uniemożliwiało pełną stabilizację mechaniczną rynny przed przyjęciem obiektu.
- Wnioski optymalizacyjne: W celu wyeliminowania błędów mechanicznych i osiągnięcia stuprocentowej skuteczności, zaleca się modyfikację funkcji sterującej `plynnyRuch`. Optymalizacja powinna polegać na programowym wydłużeniu czasu zjazdu poprzez redukcję prędkości katowej tarczy w końcowej fazie ruchu, co pozwoli na bardziej kontrolowane i płynne wprowadzenie obiektu do kanału sortującego.

6.4. Analiza specyfiki detekcji koloru czerwonego i identyfikacja błędów klasyfikacji

Następnym etapem badań była szczegółowa analiza przypadków, w których system przypisywał obiekt do kategorii „NIEZNANY”. Na podstawie przeprowadzonych testów w serii po 20 prób) ustalono, że problem ten dotyczy niemal wyłącznie koloru czerwonego. Podczas gdy pozostałe barwy osiągały 100% skuteczności rozpoznania, czerwień wykazała tendencję do wypadania poza zdefiniowane ramy decyzyjne.

Głównym powodem pomyłek była specyficzna charakterystyka sygnału odbitego od czerwonych obiektów, która znacząco różni się od pozostałych próbek:

- Niski parametr jasności całkowitej C: W przeciwieństwie do kolorów żółtego i pomarańczowego, które generują bardzo silny sygnał, gdzie parametr C często przekracza 120–200, czerwień generuje wartości w przedziale 48–55.
- Zbliżenie do tła (pusta tarcza): Wartości jasności dla czerwieni są dosyć zbliżone do odczytów pustej tarczy ($C \approx 33$), co przy drobnych wahaniach pozycji obiektu może prowadzić do uznania, że na podajniku nie ma cukierka.
- Brak dominujących cech chromatycznych: Kolor niebieski posiada bardzo silną składową B (ok. 50%), a zielony składową G (powyżej 50%), co czyni je unikalnymi. W przypadku czerwieni, choć składowa R dominuje (ok. 46–48%), to przy ogólnie niskiej intensywności sygnału, margines błędu jest bardzo wąski. Każde zakłócenie, np. matowa powierzchnia obiektu, powoduje spadek wartości poniżej progu rozpoznania.
- Podobieństwo do fioletu: Czerwień i fiolet generują zbliżone poziomy jasności, co wymusza stosowanie bardzo precyzyjnych i ciasnych warunków logicznych w kodzie Arduino, zwiększając ryzyko zakwalifikowania obiektu jako „NIEZNANY” przy najmniejszym odchyleniu.

Występowanie błędów klasyfikacji dla koloru czerwonego potwierdza, że jest on najbardziej wymagającym elementem w całym procesie sortowania. Mimo że warstwa logiczna poprawnie interpretuje większość próbek, specyfika odczytów RGB dla tej barwy sugeruje, że w przyszłości można by rozważyć zastosowanie dodatkowego doświetlenia komory pomiarowej, aby zwiększyć amplitudę sygnału odbitego dla ciemniejszych barw.

6.5. Weryfikacja spójności danych i niezawodności transmisji UART

Ostatnim etapem badań była weryfikacja integralności danych przesyłanych między urządzeniem brzegowym Arduino a aplikacją nadrzędną Python. Test ten miał na celu potwierdzenie, czy liczba i rodzaj obiektów fizycznie posortowanych przez mechanizm odpowiadają w 100% danym zapisanym w raporcie sesji oraz wyświetlanym na wykresie dynamicznym.

Metodyka i wyniki testu: Podczas wszystkich serii testowych (łącznie 120 cykli) prowadzono równoległy nadzór manualny, porównując fizyczną zawartość kubeczków z zawartością generowanego pliku raport.txt.

- Zgodność danych: Odnotowano 100% zgodności między stanem rzeczywistym a zapisem cyfrowym. Każdy sygnał wysłany przez Arduino został poprawnie odebrany, zinterpretowany i zarchiwizowany przez skrypt w języku Python.
- Brak opóźnień: Transmisja danych odbywała się w czasie rzeczywistym. Dzięki optymalizacji bufora portu szeregowego, informacja o wykrytym kolorze pojawiała się na wykresie w Pythonie natychmiast po dokonaniu pomiaru przez czujnik TCS34725, jeszcze przed fizycznym wyrzutem obiektu do rynny.
- Stabilność połączenia: Mimo długotrwałej pracy, nie stwierdzono zjawiska „gubienia ramek” ani zawieszania się interfejsu komunikacyjnego.

Wnioski: Pełna spójność danych potwierdza, że opracowana brama IoT działa w sposób niezawodny. System poprawnie steruje mechaniką i stanowi wiarygodne źródło informacji o procesie.

PODSUMOWANIE

Głównym celem niniejszej pracy inżynierskiej było zaprojektowanie oraz zbudowanie automatycznego urządzenia służącego do segregowania obiektów na podstawie ich koloru. Cel ten został w pełni osiągnięty. Realizacja projektu wymagała połączenia wiedzy z kilku dziedzin technicznych: mechaniki, elektroniki, automatyki oraz informatyki. Proces powstawania urządzenia obejmował nie tylko tworzenie oprogramowania, ale również szereg prac projektowych i konstrukcyjnych. Części mechaniczne, takie jak tarcza obrotowa, czy elementy obudowy, zostały zaprojektowane w środowisku AutoCAD, co pozwoliło na precyzyjne dopasowanie ruchomych elementów i zapewnienie płynności działania mechanizmu. Równolegle, schemat połączeń elektrycznych opracowano przy użyciu programu KiCad. Montaż fizyczny urządzenia, polegający na połączeniu serwomechanizmów, czujników oraz mikrokontrolera, wykonano ręcznie, dbając o trwałość i poprawność wszystkich połączeń.

Zbudowany prototyp funkcjonuje w modelu zbliżonym do nowoczesnych rozwiązań przemysłowych typu Internet Rzeczy (IoT), gdzie wstępne przetwarzanie danych i decyzje sterujące odbywają się bezpośrednio na urządzeniu wykonawczym. Wszystkie założone zadania projektowe zostały zrealizowane. Skonstruowano działający mechanizm podający, utworzono oprogramowanie odporne na zmiany warunków oświetleniowych oraz stworzono aplikację w języku Python. Program ten wizualizuje proces sortowania na ekranie komputera w czasie rzeczywistym i archiwizuje wyniki, co pozwala na bieżącą kontrolę pracy maszyny. System poprawnie rozpoznaje sześć zaprogramowanych kolorów i potrafi odpowiednio zareagować na stany wyjątkowe, takie jak brak obiektu na tarczy podajnika.

Kluczowym osiągnięciem w warstwie programowej było opracowanie algorytmu rozpoznawania kolorów, który opiera się na analizie wzajemnych proporcji barw. Testy przeprowadzone na seriach pomiarowych wykazały, że takie podejście zapewnia wysoką stabilność pracy układu. Badania potwierdziły, że system zachowuje poprawność klasyfikacji niezależnie od tego, czy oświetlenie w pomieszczeniu jest włączone, czy wyłączone. Jest to dowodem na skuteczność zaprojektowanej komory optycznej, która fizycznie izoluje czujnik od światła zewnętrznego. Równie istotnym wnioskiem płynącym z badań jest potwierdzenie niezawodności komunikacji między urządzeniem a komputerem. Podczas długotrwałych testów, odnotowano pełną zgodność między fizycznym stanem wyjść sortera a danymi zapisanymi w raportach cyfrowych. Oznacza to, że transmisja danych działa bezbłędnie i stabilnie.

Mimo wysokiej ogólnej skuteczności urządzenia, która w testach końcowych osiągnęła poziom ponad 98%, proces badawczy pozwolił na zidentyfikowanie pewnych ograniczeń. W nielicznych przypadkach zaobserwowano błędy sortowania wynikające z dynamiki ruchu - przy powrocie tarczy do pozycji zerowej obiekty zsuwały się zbyt szybko, co powodowało ich odbijanie się od ścianek rynny i trafiaanie do niewłaściwego pojemnika. Drugim wyzwaniem była detekcja koloru czerwonego. Ze względu na jego niską jasność w porównaniu do tła, był on trudniejszy do wykrycia niż inne, jaśniejsze barwy. Obserwacje te wskazują, że w budowie tego typu systemów konieczne jest precyzyjne dopasowanie prędkości działania mechanizmów do fizycznych właściwości sortowanych obiektów.

Zbudowany prototyp stanowi solidną bazę do dalszych prac rozwojowych. Na podstawie przeprowadzonych testów można wskazać konkretne kierunki modernizacji urządzenia. Istotnym usprawnieniem byłoby zastąpienie przewodowej komunikacji USB modulem bezprzewodowym Wi-Fi. Umożliwiłoby to przesyłanie statystyk bezpośrednio do sieci, co pozwoliłoby na zdalny podgląd pracy maszyny bez konieczności fizycznego podłączenia jej do komputera. W celu poprawy wykrywalności ciemnych kolorów, takich jak czerwony czy fioletowy, zaleca się zastosowanie dodatkowego, sterowanego programowo doświetlenia LED wewnątrz komory pomiarowej. Pozwoliłoby to na sztuczne zwiększenie kontrastu i ułatwienie pracy czujnika. Dopełnieniem tych zmian mogłaby być optymalizacja mechaniki poprzez zastosowanie szybszych napędów oraz tarczy z wieloma otworami, co pozwoliłoby na znaczące przyspieszenie procesu sortowania przy jednoczesnym wyeliminowaniu problemu bezwładności obiektów. Podsumowując, zrealizowana praca udowodniła, że przy wykorzystaniu dostępnych podzespołów możliwe jest zbudowanie funkcjonalnego i precyzyjnego systemu automatyki.

LITERATURA

1. Arduino.cc, *Arduino Nano Documentation – Hardware and Specifications*, 2024. [Online]. Dostępne: <https://docs.arduino.cc/hardware/nano> [dostęp: 05.01.2026].
2. Binder+Co, *CLARITY Sorting Machines: Recycling Systems*, 2024. [Online]. Dostępne: <https://www.binder-co.com/en/products/clarity-sorting-machines/clarity-recycling-systems/> [dostęp: 06.12.2025].
3. Botland, *Wyświetlacz LCD 2x16 znaków niebieski + konwerter I2C LCM1602*, 2024. [Online]. Dostępne: <https://botland.com.pl/wyswietlacze-alfanumeryczne-i-graficzne/2351-wyswietlacz-lcd-2x16-znakow-niebieski-konwerter-i2c-lcm1602-5904422309244.html> [dostęp: 05.01.2026].
4. Bühler Group, *SORTEX Optical Sorter Technical Brochure*, 2023. [Online]. Dostępne: <https://www.buhlergroup.com> [dostęp: 06.12.2025].
5. Cimbria, *SEA XL – Optical Sorting*, 2025. [Online]. Dostępne: <https://www.cimbria.com> [dostęp: 06.12.2025].
6. *Dokumentacja techniczna modułu zasilania impulsowego DC/DC 5V/3A z funkcją UPS*, 2024. [Online]. Dostępne u dostawcy podzespołów elektronicznych [dostęp: 06.01.2026].
7. Groover, M. P., *Automation, Production Systems, and Computer-Integrated Manufacturing*, 5th ed. Upper Saddle River, NJ: Pearson, 2020. [Online]. Dostępne: <https://www.pearson.com/en-us/subject-catalog/p/automation-production-systems-and-computer-integrated-manufacturing/P200000003254/9780134605463> [dostęp: 06.12.2025].
8. Jameco Electronics, *How Servo Motors Work*, 2024. [Online]. Dostępne: <https://www.jameco.com/Jameco/workshop/Howitworks/how-servo-motors-work.html> [dostęp: 21.01.2026].
9. Kasprzyk, J., *Programowanie sterowników przemysłowych*, Gliwice: Wydawnictwo Politechniki Śląskiej, 2006. [Online]. Dostępne: <https://www.scribd.com/document/762180834/Kasprzyk-Jerzy-Programowanie-Sterownikow-Przemys%C5%82owych> [dostęp: 06.12.2025].

10. Key Technology, *Integrated Processing Solutions, Technical Brochure*, 2022. [Online]. Dostępne: <https://www.key.net/wp-content/uploads/2022/08/Integrated-Processing-Solutions-US-Pages-180427.pdf> [dostęp: 06.12.2025].
11. Kostro, J., *Elementy, urządzenia i układy automatyki*, wyd. 5, Warszawa: Wydawnictwa Szkolne i Pedagogiczne, 2013. [Online]. Dostępne: <https://www.scribd.com/document/784804336/Kostro-J-Elementy-Urz%C4%85dzenia-i-Uk%C5%82ady-Automatyki> [dostęp: 06.12.2025].
12. Koul, P., *Automation and Industry 4.0 in Production Engineering: A Comprehensive Review*, „Advances in Mechanical and Materials Engineering”, vol. 42, no. 1, pp. 149–190, 2025. [Online]. Dostępne: <https://journals.prz.edu.pl/amme/article/view/1862> [dostęp: 21.01.2026].
13. Kushner, D., „The Making of Arduino”, *IEEE Spectrum*, 2011. [Online]. Dostępne: <https://spectrum.ieee.org/the-making-of-arduino> [dostęp: 06.12.2025].
14. LEGO Education, *LEGO Mindstorms EV3 Color Sorter: Educational Guide and Technical Specifications*, 2024. [Online]. Dostępne: <https://education.lego.com> [dostęp: 04.01.2026].
15. Malucha, M., „Internet rzeczy – kontekst technologiczny i obszary zastosowań”, *Studia i Prace WNEiZ US*, nr 54/2, s. 51–69, 2018. [Online]. Dostępne: <https://wnus.usz.edu.pl/sip/file/article/download/15260.pdf> [dostęp: 25.01.2026].
16. Medojevic, I., Veg, E., Joksimovic, A. i Ilic, J., *Promotion of Color Sorting in Industrial Systems Using a Deep Learning Algorithm*, „Applied Sciences”, vol. 12, no. 24, art. no. 12817, 2022. [Online]. Dostępne: <https://www.mdpi.com/2076-3417/12/24/12817> [dostęp: 06.12.2025].
17. Meyer Europe, *Sorter optyczny Meyer M2 – MSXC-120M: Specyfikacja techniczna*, 2024. [Online]. Dostępne: <https://meyer-corp.eu/pl/sortowniki-optyczne/m2/> [dostęp: 04.01.2026].
18. Muc, A., Murawski, L., Gesella, G., Szeleziński, A., Szarmach, A., „Zastosowanie platform cyfrowych Arduino i Raspberry Pi w nauczaniu sterowania obiektem pneumatycznym”, *Zeszyty Naukowe Wydziału Elektrotechniki i Automatyki Politechniki Gdańskiej*, nr 41, s. 47–50, 2015. [Online]. Dostępne: <https://yadda.icm.edu.pl/baztech/element/bwmeta1.element.baztech-5c123e6f-413a-4a30-8405-051efc5f50bd> [dostęp: 06.12.2025].

19. Philips Semiconductors, *PCF8574; Remote 8-bit I/O expander for I2C-bus - Product specification*, 1997.
20. Satake Corporation, *Evolution of Optical Sorting Technology for Grains and Beans*, 2024. [Online]. Dostępne: <https://www.satake-group.com> [dostęp: 06.12.2025].
21. TOMRA Systems ASA, *Leading the Resource Revolution: Sorting Solutions Technical Manual*, 2024. [Online]. Dostępne: <https://www.tomra.com> [dostęp: 06.12.2025].
22. TowerPro, *MG90S Micro Servo Specification and Dimensions*, 2024. [Online]. Dostępne: <http://www.towerpro.com.tw/product/mg90s-3/> [dostęp: 05.01.2026].
23. Waveshare Wiki, *TCS34725 Color Sensor – User Manual and Specification*, 2024. [Online]. Dostępne: https://www.waveshare.com/wiki/TCS34725_Color_Sensor [dostęp: 05.01.2026].
24. WESORT (Zhongke Weishideng), *Coffee Bean Color Sorter – Technical Specification*, 2024. [Online]. Dostępne: <https://www.wesortcoloursorter.com/product/coffee-bean-color-sorter-7/> [dostęp: 04.01.2026].

ZAŁĄCZNIKI

Spis rysunków

Rys. 1.1.1. Przemysłowy system sortowania optycznego AUTOSORT

Rys. 1.1.2. Widok techniczny sortera optycznego z serii SORTEX A

Rys. 1.1.3. Przemysłowy sorter optyczny wykorzystujący wysokociśnieniowe dysze pneumatyczne

Rys. 1.1.4. Schemat wewnętrzny i zasada działania kaskadowego sortera optycznego serii SEA

Rys. 1.2.1. Rysunek zestawieniowy sortera optycznego WESORT z określeniem głównych wymiarów gabarytowych

Rys. 1.3.1. Model edukacyjny systemu sortującego kolory oparty na platformie LEGO Mindstorms EV3

Rys. 2.2.1. Moduł czujnika koloru TCS34725 z widocznym filtrem IR oraz diodą doświetlającą

Rys. 2.2.1. Moduł zaawansowanego czujnika koloru TCS34725

Rys. 2.2.2. Przebiegi czasowe sygnałów startu oraz stopu na magistrali I2C

Rys. 2.3.1. Schemat wymiarowy serwomechanizmu MG-90S

Rys. 2.4.1. Moduł wyświetlacza LCD 16x2 z konwerterem I2C

Rys. 2.5.1. Moduł impulsowej przetwornicy DC-DC z funkcją UPS

Rys. 3.1.1. Zestawienie kolorystyczne drażetek

Rys. 3.2.1. Schemat blokowy algorytmu sterującego

Rys. 3.3.1. Prototyp funkcjonalny wykonany w celu weryfikacji założeń mechanicznych

Rys. 3.3.3. Zrzut ekranu z programu AutoCAD przedstawiający trójwymiarowe modele części

Rys. 3.4.1. Schemat blokowy architektury elektronicznej systemu

Rys. 3.4.2. Wstępny prototyp układu sterowania wykonany na płycie stykowej.

Rys. 3.4.3. Schemat ideowy układu w programie KiCad z widocznymi kondensatorami i złączami modułów

Rys. 3.4.4. Projekt ścieżek na warstwie dolnej PCB (B.Cu)

Rys. 3.4.5 Wizualizacja 3D górnej warstwy płytki z obsadzonymi komponentami

Rys. 3.4.6. Wizualizacja 3D dolnej warstwy płytki przedstawiająca prowadzenie ścieżek

Rys. 4.1.1. Zestaw płaskich elementów obudowy po zakończeniu procesu druku 3D

Rys. 4.1.2. Proces montażu i klejenia korpusu urządzenia z wykorzystaniem ścisków stolarskich

Rys. 4.1.3. Gotowa płytka sterująca PCB z przylutowanymi gniazdami modułów oraz zestawem kondensatorów filtrujących zasilanie.

Rys. 4.3.1. Weryfikacja odczytów kolorometrycznych w selektorze barw Google dla próbek koloru zielonego (RGB: 46, 103, 46)

Rys. 4.5.1. Kompletny sorter do cukierków Skittles - widok od przodu.

Rys. 4.5.2. Kompletny sorter do cukierków Skittles - widok modułu wyświetlającego.

Rys. 4.5.3. Kompletny sorter do cukierków Skittles - widok z tyłu, eksponujący zasilanie oraz sposób podłączenia urządzenia do płytki PCB.

Rys. 5.1.1. Schemat blokowy przepływu danych w architekturze Edge-Gateway.

Rys. 5.4.1. Interfejs graficzny aplikacji monitorującej z wynikami końcowymi sesji.

Rys. 5.4.1. Przykładowy raport sesji wygenerowany w pliku tekstowym po zatrzymaniu programu.

Spis listingów

Listing 4.2.1. Program do odczytu surowych danych kolorometrycznych

Listing 4.4.1. Definicja stanów pracy oraz konfiguracja bibliotek i zmiennych.

Listing 4.4.2. Konfiguracja startowa, autotest systemu i kalibracja mechaniczna (Setup).

Listing 4.4.3. Implementacja autorskiej funkcji sterowania ruchem (plynnyRuch).

Listing 4.4.4. Moduł statystycznej obróbki sygnału i uśredniania danych z sensora.

Listing 4.4.5. Implementacja algorytmu wyboru koloru i logiki decyzyjnej (if-else)

Listing 4.4.6. Procedura obsługi zatorów, braku materiału i blokady systemowej.

Listing 4.4.7. Funkcja aktualizacji interfejsu lokalnego i statystyk na wyświetlaczu LCD.

Spis tabel

Tabela 4.3.1 Przykładowe pomiary seryjne dla barwy zielonej odczytane za pomocą czujnika.

Tabela 4.3.2. Zestawienie średnich wartości składowych dla analizowanych barw i stanów układu.

Tabela 6.1.1. Wyniki testów funkcjonalnych podzespołów (seria 50 powtórzeń).